FINAL PROJECT REPORT

Consolidated (DA1, DA2, DA3)

NeuroSonic: From a Cross-Modal Wearable Obstacle Detector to an Offline, Phone-Based Spoken Walking-Path Awareness and Navigation Prototype for Visually Impaired Pedestrians

Submitted in partial fulfilment of the requirements of

MACSE517N – COMPUTER ARCHITECTURE AND ORGANISATION

by

ALLWIN RAJ R (26MAI0072)

MADHU SUDHARSHANAN (26MAI0071)

M.Tech. Computer Science and Engineering (Artificial Intelligence and Machine Learning)

Under the guidance of

YOGESH BABU S

Associate Professor Grade 1

School of Computer Science and Engineering

VELLORE INSTITUTE OF TECHNOLOGY

Vellore, Tamil Nadu, India · November 2026

Live demo video

https://drive.google.com/file/d/1yJS2f8_ee3ZwV55mkA9e1c6mm9f5xkYd/view?usp=sharing

Declaration

We declare that this report describes work done by us. Results reported as measured were measured by us on the equipment described; results reported by other authors are cited and labelled as such. Physical validation with an iPhone in real environments and with users with visual impairment has been carried out only informally (Section 18.7) and the formal validation has not been completed, so no statement in this report should be read as a claim that the system is safe to rely on.

Place: Vellore
Date: 3 November 2026

Allwin Raj R (26MAI0072)
Madhu Sudharshanan (26MAI0071)

Abstract

Pedestrians with visual impairment need to know both what is in front of them now and where to turn. This report consolidates three reviews of one project. Review 1 substantiates the problem with WHO, Global Burden of Disease and Indian survey statistics (at least 2.2 billion people with vision impairment; 43.3 million blind in 2020 and 61 million projected by 2050; 1.99 % blindness among Indians aged 50 and over) and compares published systems, nine verified patents and eight commercial products, followed by five technology trends. Review 2 re-implements the base paper, WOAD (Gao et al., Nature Communications, 2025), a near-infrared + depth cross-modal detector. The standalone re-implementation is bit-identical to the original, but the reported accuracy cannot be reproduced with the published data, the cross-dataset person AP@0.5 is 0.218, detections collapse on darker sensor recordings, and the model cannot be driven by a phone camera. Eight gaps are derived. Review 3 presents NeuroSonic, in which an iPhone browser is the camera, microphone, GPS receiver and speaker of a system that runs offline on a laptop GPU. RF-DETR Nano, selected among four detectors for its person AP@0.5 (0.660) and recall (0.729), feeds a camera-motion-compensated tracker that announces people or vehicles entering or crossing the walking path without reacting to the user's own movement. Offline neural speech, voice commands and OpenStreetMap routing complete the system. On the laptop it sustained about 5 frames/s with a median capture-to-answer time of 67 ms; three serving problems were found and fixed; ten-minute soak tests ran without errors; 204 automated tests and 7 browser tests pass. Live screenshots from an iPhone show the guidance banners and status line working at 2.5–4.8 frames/s with 217–372 ms from camera to answer. Formal field validation with users is pending; the protocol for it is included. The system is a research prototype and not a safety device.

Keywords: assistive technology; visual impairment; obstacle detection; RF-DETR; multi-object tracking; camera-motion compensation; offline speech; pedestrian navigation; reproducibility.

Table of Contents

Chapter 1: Introduction: Domain and Field of Work **6**

Chapter 2: Problem Substantiation with Statistics **8**

2.1 Global burden 8

2.2 The situation in India 9

2.3 What the statistics imply for this project 10

Chapter 3: Survey: Comparative Analysis of Existing Solutions (Publications) **11**

3.1 Method of the survey 11

3.2 Review papers 11

3.3 Wearable depth-camera systems 11

3.4 Robot guides and segmentation approaches 12

3.5 Observations 13

Chapter 4: Survey: Comparative Analysis of IPRs (Patents) **14**

4.1 Search method 14

4.2 The patents 14

4.3 Observations from the patent survey 15

Chapter 5: Survey: Comparative Analysis of Company Products **17**

5.1 Smart canes 17

5.2 Wearable cameras 17

5.3 Smartphone apps 17

5.4 Harness and robotic guides 17

5.5 Observations from the product survey 18

Chapter 6: Technology Trend: Introduction and Adaptation Status **19**

6.1 From range sensing to recognition 19

6.2 Transformer-based detection 19

6.3 Multi-modal and depth sensing 19

6.4 Edge, offloading and dedicated hardware 20

6.5 Vision-language models and offline speech 20

Chapter 7: Synthesis and Conclusion of the Survey **21**

Chapter 8: The Base Paper: WOAD **22**

8.1 System overview 22

8.2 Detector architecture 22

8.3 Results reported by the authors 23

Chapter 9: Re-implementation of the Base Paper **24**

9.1 What was released, and its state 24

9.2 Faithful re-implementation 24

9.3 Dataset audit 24

9.4 Evaluation method 25

9.5 Results 25

9.6 Brightness and live-input experiment 27

9.7 Re-use of the base model in this project 27

Chapter 10: Reproduction Procedure and Repository Audit....28
10.1 Components of the released work....28
10.2 Procedure....28
10.3 Download links....28
Chapter 11: Threats to Validity of the Re-implementation....30
Chapter 12: Building on the Base Model: Alert Engine and Input-Quality Gate....31
Chapter 13: Cons of the Base Work and Gap Identification....32
13.1 Cons observed....32
Chapter 14: Proposed Work....34
14.1 Problem statement....34
14.2 Objectives....34
14.3 Proposed architecture....34
14.4 Methodology....35
14.5 Evaluation plan....36
14.6 Differentiation from the base paper....36
Chapter 15: System Architecture....37
15.1 Deployment....37
15.2 Phone client....38
15.3 Laptop server....38
15.4 Privacy and safety by design....38
Chapter 16: Methodology....39
16.1 Detector selection....39
16.2 Detector runtime and fall-back....39
16.3 Camera motion estimation....40
16.4 Motion-compensated tracking....40
16.5 Walking-path states and events....40
16.6 Speech, voice commands and routing....41
Chapter 17: Experimental Setup....43
Chapter 18: Results and Analysis....44
18.1 Detector comparison....44
18.2 FP16 inference....46
18.3 Serving efficiency at a fixed 5 frames/s....46
18.4 Ten-minute soak tests....47
18.5 End-to-end behaviour in the browser....48
18.6 Automated tests and quality gate....49
18.7 iPhone screenshots....49
18.8 Live readings from the iPhone....51
Chapter 19: Discussion....53
Chapter 20: Limitations and Pending Validation....54
Chapter 21: Conclusion....55
Chapter 22: Project Demonstration: Existing Implementation and Proposed Work....56

22.1 Demonstration script (about 10 minutes)..... 57
Appendix A: API Endpoints..... 58
Appendix B: Software, Models and Licences..... 59
Appendix C: Installation, Launch and Reproduction..... 60
Appendix D: Physical Validation Protocol (Summary)..... 61
References..... 62

Chapter 1: Introduction: Domain and Field of Work

This work belongs to *assistive technology for people with visual impairment*, and within it to the sub-field of *electronic travel aids* (ETAs) and *navigation aids*. An ETA is a device that senses the environment ahead of a pedestrian and turns that information into a signal the person can hear or feel. A navigation aid tells the person where they are and which way to turn to reach a destination. The two needs are related but different: the first is about the next few metres and the next few seconds, the second about the whole journey [1], [2].

For decades the white cane and the guide dog have been the primary mobility aids. The cane is cheap, reliable, and gives immediate physical feedback about the ground, but its range is roughly the length of the cane, it does not detect obstacles at head or chest height, and it does not tell the user what an object is or whether it is moving. Guide dogs address some of these limits but are scarce and costly to train. The base paper of this project reports that, in its own trials, users with a white cane avoided only about a quarter of suddenly appearing indoor obstacles, and about 40 % of fast-moving cars at night [3]. These figures come from a specific experimental set-up, but they illustrate why supplementary sensing is studied at all.

The field draws on several engineering disciplines at once:

- **Sensing:** ultrasonic range finders, infrared and time-of-flight (ToF) depth cameras, ordinary RGB cameras, inertial sensors and satellite positioning (GNSS/GPS) [1].
- **Perception:** object detection, semantic and panoptic segmentation, tracking and scene description, today dominated by deep neural networks [4], [5], [6].
- **Human–computer interaction:** how to present information through hearing (speech, spatial audio, tones) or touch (vibration, braille) without overloading a sense that the user already relies on to move safely [1], [7].
- **Embedded and edge computing:** running perception within the power, weight, latency and cost limits of a wearable or a phone, sometimes with dedicated hardware such as FPGAs [3].
- **Navigation:** maps, routing and localisation, both outdoors (GNSS) and indoors (beacons, visual odometry) [8], [9].

Over the last decade, three changes have reshaped the field. First, deep-learning detectors made it possible to recognise many object classes in real time on modest hardware [4], [10], [11]. Second, smartphones became capable sensing and computing platforms that most users already carry, which lowers the cost and the social stigma of a dedicated device [2]. Third, large vision-language models have made free-form scene description available as a cloud service, which is now offered in consumer apps for blind users [12].

The present project is called **NeuroSonic**. It started from the base paper *WOAD: a wearable obstacle avoidance device for visually impaired individuals with cross-modal learning*, published in *Nature Communications* in 2025 [3]. WOAD combines self-developed glasses with a near-infrared camera and a ToF depth sensor, a reinforcement-learning video compression module, an FPGA, and a smartphone that runs a cross-modal (infrared + depth) obstacle detector. NeuroSonic re-implements and evaluates the WOAD detector, identifies the gaps that prevent it from being used with ordinary phones, and proposes and implements a phone-camera system with spoken guidance that runs offline on a laptop. This work first surveys the domain, the size of the problem, publications, patents, products and technology trends; it then re-implements the base paper and identifies its gaps; finally it presents the design, implementation and evaluation of the proposed system.

The scope of the survey is deliberately bounded. It covers systems that sense the environment in front of a walking person and communicate with the person in real time, plus the navigation aids they are combined with. It does not cover reading aids (text recognition for documents) except where they are part of the same product, and it does not cover medical treatment of vision loss. The patents and products listed are those that could be verified from primary pages (patent office records, publisher pages, manufacturer or established-retailer pages); they are a sample of the field, not an exhaustive census.

Chapter 2: Problem Substantiation with Statistics

2.1 Global burden

The World Health Organization (WHO) estimates that, globally, at least **2.2 billion people** have a near or distance vision impairment, and that for at least **1 billion** of them the impairment could have been prevented or has yet to be addressed [13]. The 2.2-billion figure was first published in WHO's *World Report on Vision* in October 2019 [14]. Among people with distance vision impairment or blindness, the leading causes are cataract (94 million), refractive error (88.4 million), age-related macular degeneration (8 million), glaucoma (7.7 million) and diabetic retinopathy (3.9 million); near vision impairment is mainly due to presbyopia (826 million) [13]. Fig. 1 shows the distance-vision causes.

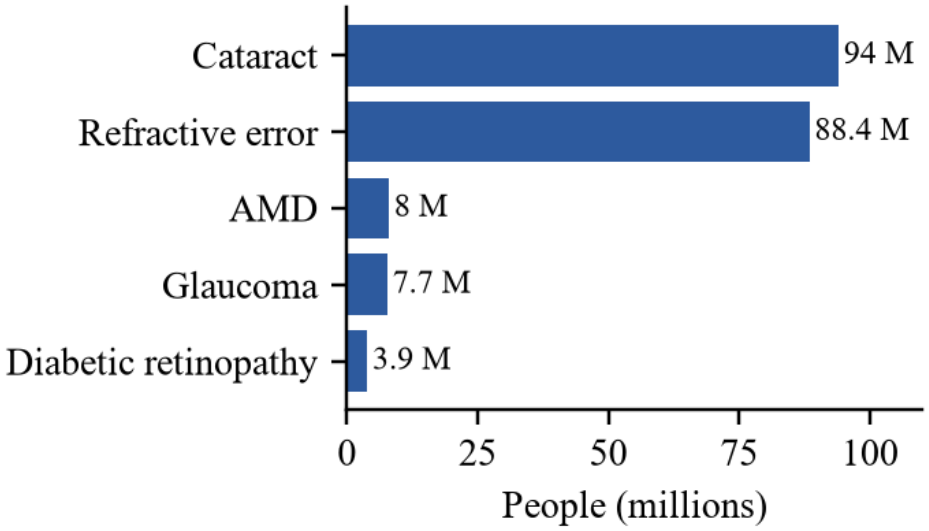


Fig. 1. *Leading causes of distance vision impairment or blindness worldwide, in millions of people (WHO fact sheet, February 2026 [13]).*

Description. The bar chart ranks the main causes of distance vision impairment worldwide. Uncorrected refractive error and cataract account for most cases, which are largely avoidable or treatable. The chart supports the project's premise that a very large group of pedestrians has no reliable way to see what is ahead of them.

The Global Burden of Disease (GBD) Vision Loss Expert Group estimated that in 2020, **43.3 million** people were blind (95 % uncertainty interval 37.6–48.4 million), of whom 23.9 million (55 %) were female, and that 295 million people had moderate or severe vision impairment [15]. Between 1990 and 2020 the age-standardised prevalence of blindness among adults aged 50 years and older fell by 28.5 %, yet the *number* of blind people rose by 50.6 %, because populations grew and aged. The same study projects **61.0 million** blind people and 474 million with moderate or severe vision impairment by 2050 [15] (Fig. 2). In other words, medical progress is reducing the risk for each person, but the absolute number of people who need mobility support is still increasing.

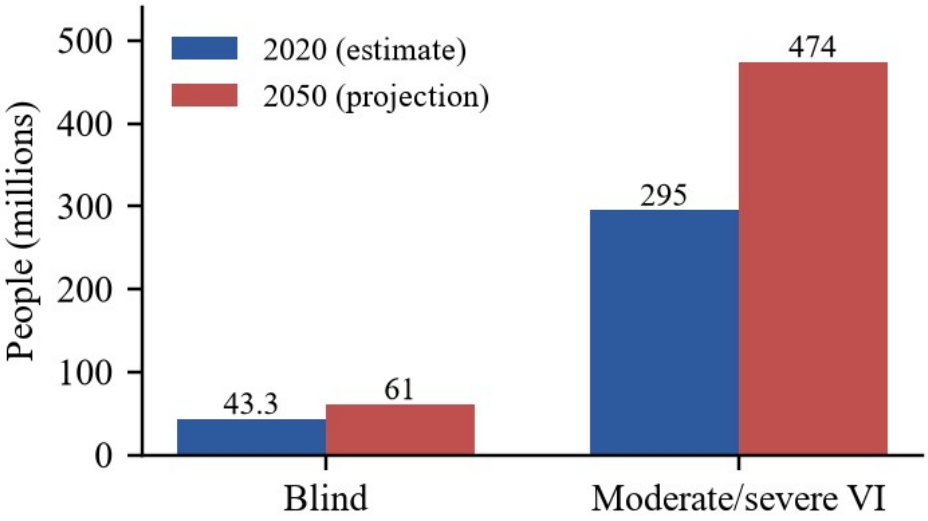


Fig. 2. *People who are blind or have moderate/severe vision impairment (VI): 2020 estimate and 2050 projection, millions (GBD study [15]).*

Description. The paired bars compare the number of people who are blind or have moderate or severe vision impairment in 2020 with the 2050 projection. The growth between the two bars shows that the problem is increasing, so assistive tools that need only a phone become more valuable over time.

2.2 The situation in India

India's National Blindness and Visual Impairment Survey 2015–2019, run under the National Programme for Control of Blindness and Visual Impairment with AIIMS, surveyed people aged 50 years and older in 31 districts [16]. It reports a national prevalence of blindness of **1.99 %** in this age group, with cataract the major cause (66.2 % of blindness), and a 47.1 % reduction in blindness compared with the 2010 baseline [16], [17]. Prevalence rises steeply with age: 0.5 % at 50–59 years, 1.6 % at 60–69, 4.1 % at 70–79 and 11.6 % at 80 years and above [17] (Fig. 3). District values vary considerably, which matters for any local deployment [16].

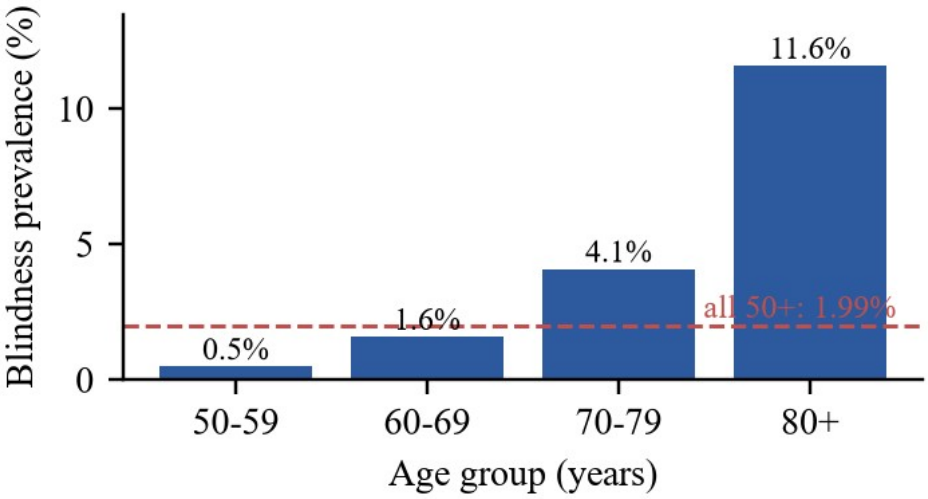


Fig. 3. *Prevalence of blindness by age group among people aged 50+ in India (NBVIS 2015–2019 [17]). Dashed line: overall 50+ prevalence, 1.99 % [16].*

Description. The bars show how blindness prevalence rises with age among Indians aged 50 and over, with the dashed line marking the overall rate of 1.99 %. Older pedestrians are affected most, which motivates simple spoken guidance that does not need touching a screen.

2.3 What the statistics imply for this project

Table 1. *Key statistics and their relevance to the problem*

Statistic	Value	Source	Implication
People with near or distance VI	≥ 2.2 billion	[13]	very large population; low-cost devices matter
Blind people (2020)	43.3 million	[15]	core users of mobility aids
Blind people projected (2050)	61.0 million	[15]	demand will grow
Blindness prevalence, India, age 50+	1.99 %	[16]	older users; simple interaction needed
Blindness prevalence, India, age 80+	11.6 %	[17]	age-friendly design (speech, few controls)
White-cane avoidance of sudden indoor obstacles (base-paper trial)	≈ 26 %	[3]	supplementary sensing can help

Three implications follow. *First*, the user population is large, growing and old. A device for this population must be affordable and must be operable by people who may not be familiar with technology; speech is the most natural channel. *Second*, most of these users live in low- and middle-income countries, where the cost of dedicated hardware (several thousand US dollars for some commercial wearables, see the product survey) is a serious barrier, whereas a smartphone is often already owned. *Third*, because vision impairment is often partial and acquired late in life, people continue to rely on the cane. An electronic aid should supplement, not replace, the cane, and must never claim more than it can deliver. This last point is central to the design principles of NeuroSonic: it never states distances, never says that a way is "safe", and never claims 360-degree awareness.

The statistics do not, by themselves, measure the frequency of collisions or falls caused by obstacles while walking; we did not find a recent, population-level Indian statistic on that question that we could verify, so we do not quote one. The base paper's comparison with the white cane [3] is an experimental result from twelve participants, not a population statistic, and is quoted only as an illustration.

Chapter 3: Survey: Comparative Analysis of Existing Solutions (Publications)

3.1 Method of the survey

Publications were selected to cover the main families of approaches: (i) review papers that map the field; (ii) wearable RGB-D or ToF systems with on-body or phone processing; (iii) robot guides; (iv) segmentation-based "walkable area" methods; and (v) the base paper. Each was checked against the publisher, PubMed Central or arXiv record. Where only the abstract was accessible, we report only what the abstract states and do not infer numbers. The comparison criteria are: *sensors*, *where the computation runs*, *perception method*, *user feedback*, *navigation support*, *evaluation*, and the *main limitation* relevant to a low-cost, phone-based system.

3.2 Review papers

Real and Araujo [1] review navigation systems for the blind from the first sonar-based ETAs and sensory substitution devices to smartphone, wearable and cloud-based designs. They identify recurring challenges: a heterogeneous user population, the limited bandwidth of hearing and touch, high cognitive load, poor acceptance of past devices, and a trade-off between positioning accuracy and infrastructure cost. Kuriakose, Shrestha and Sandnes [2] survey tools and technologies for navigation support and give design recommendations for future systems. Both reviews stress that technical performance alone does not guarantee adoption, which is why user-facing behaviour (what is said, when, and how reliably) is treated as a first-class requirement in this project.

3.3 Wearable depth-camera systems

WOAD (base paper) [3] is the most complete recent system. Self-developed glasses (about 400 g including an 80 g battery) carry a Synexens CS30 sensor with a near-infrared camera and a ToF depth sensor (640×480, 0.1–5 m). A depth-aided compression module, whose policy is learned with a discrete soft actor-critic reinforcement-learning agent [18], decides which frames to send and at what quality; it runs on a custom FPGA board. A smartphone (Samsung Galaxy S23 Ultra) runs a YOLOv5-based [19] detector extended with a second input stream and transformer blocks [20] that fuse infrared and depth features ("cross-modal learning"). Alerts are spoken (attention/warning/danger) and given by three vibration motors that encode direction and distance. The authors report 100 % collision avoidance in their indoor and outdoor scenarios, an end-to-end delay below 320 ms, about 11 h of operation, and a System Usability Scale score of 83.75 with twelve participants providing quantitative data [3]. It needs the custom glasses and the CS30 sensor.

Bai et al. [21] built a glasses-shaped travel aid with a consumer RGB-D camera and an inertial measurement unit. Ground segmentation separates the walkable ground from obstacles; a CNN on a smartphone classifies objects. The walkable direction is conveyed by beeps and object categories by speech. Twenty visually impaired participants navigated an office environment and avoided collisions; the abstract gives no quantitative accuracy or latency [21]. **Lin et al.** [22] combine an RGB-D wearable terminal, an earphone and a smartphone touch interface with a separate, powerful processor; a learned model predicts walkable instructions from RGB-D data and a semantic map. They report good results on indoor and outdoor datasets with low-lying obstacles and a user study, without numbers in the abstract.

Xu, Song and Wang [23] present a head-mounted obstacle-avoidance wearable (an extended version of a 2023 conference paper). **Tang et al.** [7] describe a human-centred wearable assistance system: an RGB-D camera on a 3D-printed glasses frame, ultrathin "artificial skins" with haptic actuators on the wrist and fingers, a bone-conduction earphone and smart insoles, with all computation local. It was tested with humanoid robots and with 20 visually impaired participants without prior experience of such systems; the authors argue that human adaptation has been under-considered in earlier systems [7]. **Yan et al.** [24] propose gait-based guidance using multimodal sensing and report better performance than a guide cane in indoor and outdoor experiments (preprint; no numbers in the abstract). **AviPer** [25] uses a visual-tactile multimodal attention network.

3.4 Robot guides and segmentation approaches

CaBot [26] is a suitcase-shaped autonomous navigation robot that guides blind users to destinations in unfamiliar indoor environments while avoiding obstacles, with a vibro-tactile handle; it was evaluated with ten blind users. It moves the sensing burden from the person to a robot, at the cost of carrying a robot. **Ghahremanians and Mahvash Mohammadi** [6] take a segmentation view: instead of detecting individual obstacles, a panoptic network finds the safe walking area and treats everything else as an obstacle. Their model reaches a panoptic quality of 37.6 on Cityscapes at about 4.3 frames/s (232 ms) on an RTX 4090, trained on a custom dataset that is not public.

Table 2. *Comparative analysis of published solutions*

Work	Sensors	Compute	Perception	Feedback	Navigation	Evaluation (as reported)	Main limitation for a low-cost phone system
WOAD [3] (base)	NIR + ToF depth (CS30) on glasses	FPGA + smartphone	YOLOv5 + transformer cross-modal fusion	speech levels + 3 vibration motors	no route guidance	12 users; 100 % avoidance; <320 ms; ≈11 h	custom glasses and CS30 sensor required
Bai et al. [21]	RGB-D + IMU, glasses	smartphone	ground segmentation + CNN	beeps + speech	walkable direction	20 users, office	depth camera required
Lin et al. [22]	RGB-D wearable	separate processor + phone	learned walkable instructions, semantic map	earphone, touch	local instructions	datasets + user study	extra processor and depth camera
Xu et al. [23]	head-mounted sensors	wearable	obstacle detection	wearable alerts	—	see paper	dedicated device
Tang et al. [7]	RGB-D, artificial skins, insoles	local, on device	multimodal perception	bone conduction + haptics	guidance	robots + 20 users	several custom wearables
CaBot [26]	robot sensors	robot	autonomous navigation	vibro-tactile handle	indoor destinations	10 blind users	robot must be carried; indoor
Ghahremanians [6]	camera (smart glasses)	desktop GPU	panoptic segmentation of walkable area	—	—	PQ 37.6; 232 ms	no user study; dataset not public
NeuroSonic (this work)	phone RGB camera, mic, GPS	laptop GPU over Wi-Fi	RF-DETR + motion-compensated tracking + events	neural speech	offline map + A* routing	laptop tests only (no users yet)	needs laptop; no depth; not field-validated

3.5 Observations

- **Depth is the norm in research systems.** Nearly all wearable systems above use RGB-D or ToF sensors, which measure distance directly. A phone RGB camera does not, so a phone-based system must either avoid distance claims or estimate depth by other means.
- **Custom hardware limits reach.** Glasses, FPGAs, artificial skins and robots deliver strong results in studies but are not available to most users, especially in India.
- **Obstacle detection and navigation are rarely combined.** Most systems focus on the next few metres; route guidance is handled separately (e.g. by phone map apps), if at all.
- **Motion semantics are under-explored.** The surveyed detectors work frame by frame. Telling the user that a person is *moving into* the path, while not confusing this with the user's own walking, requires tracking and camera-motion compensation [27], [28].
- **Reproducibility is uneven.** Several works report results on private datasets; we found in re-implementing the base paper that its published dataset differs from the one behind its reported accuracy (see the base-paper re-implementation).

Chapter 4: Survey: Comparative Analysis of IPRs (Patents)

4.1 Search method

Patents were searched on Google Patents with combinations of the terms *visually impaired*, *blind*, *obstacle detection*, *navigation*, *camera*, *audio* and *wearable*, and in the CPC group A61H 3/06 (walking aids for blind persons). For each candidate the bibliographic record was opened and the assignee, inventors, dates, legal status and summary were read. Nine documents were retained, chosen to span four decades and the main technical families. A search of the Indian Patent Office database was not completed in this review; for example, for IIT Delhi's SmartCane we could confirm the device and its funding but not a patent number, so it appears under products instead. A professional freedom-to-operate search would be needed before any filing (an invention disclosure has been prepared alongside this work).

4.2 The patents

US 4 280 204 (Polaroid, 1981) [29] describes a mobility cane with a capacitive electrostatic ultrasonic transducer that both emits and receives; one mode warns of low-hanging objects at upper-body height and another detects more distant objects. It is an early example of the ultrasonic ETA family, expired long ago.

US 2005/0208457 A1 (Fink and Humayun; later assigned to Caltech) [30] proposes a head- or eyeglass-mounted camera that captures an image on request, classifies the object with neural networks, look-up tables or thresholds, and speaks a description including colour and brightness. The application was abandoned. It anticipates, in a simple form, the camera-plus-speech idea of today's apps.

US 9 429 446 B1 (Searle et al., 2016) [31] claims a hand-held device that locates objects with a camera, radio, sonar or infrared module, plans a path and gives spoken directions. It lapsed in 2024 for non-payment of maintenance fees.

US 9 846 231 B2 (STMicroelectronics, 2017) [32] claims a wearable obstacle detector with an ultrasonic source and receiver plus a three-axis accelerometer; the control module enables the ultrasound and the time-of-flight measurement only when the foot rests on the ground during walking, then computes the obstacle distance and raises an alarm within a given range. The idea of gating sensing by gait phase is relevant to camera systems too, because image quality and apparent motion also depend on the walking cycle.

US 9 993 384 B1 (Toyota, 2018) [33] guides a user towards two destinations simultaneously with two distinct audio cues whose pitch or volume varies with the alignment between the user's heading and each path.

US 10 299 982 B2 (Frankel, 2019) [34] uses a smartphone or tablet camera and orientation sensors to build a 3-D model of a room; touching a spot on a top-down view announces the object there and its attributes, such as potential hazards. Expired in 2023.

US 10 809 079 B2 (Ford, 2020) [35] describes a wearable garment such as a vest with GPS receivers, cameras, an ultrasonic sensor and a microphone that detect static and dynamic obstacles to generate and revise routes, with audio instructions and haptic cues, and optional upload of dynamic-obstacle data to update a map.

US 11 112 261 B2 (Brown University, 2021) [9] covers wearable navigation devices with multiple fixed cameras that localise the user by matching features against prior images and by visual-inertial odometry, and haptic belts or vests that render directional cues.

US 11 318 050 B2 (American Printing House for the Blind, 2022) [8] uses a smartphone's location services (GPS, compass, accelerometer, gyroscope) to narrate only the streets and points of interest in the direction the device is pointed, and Bluetooth beacons for indoor position and floor, with audio and haptic or braille output.

Table 3. *Comparative analysis of patents*

Patent	Assignee	Year (grant/publ.)	Status*	Sensing	Processing / key idea	Output	Relation to NeuroSonic
US 4,280,204 [29]	Polaroid	1981	expired	ultrasonic transducer on cane	two range modes (upper body / distant)	warning signal	range sensing only; no recognition
US 2005/0208457 [30]	Caltech (assignee of record)	2005	abandoned	head-mounted camera	object classification on request	speech description	single images on demand; no tracking or path logic
US 9,429,446 [31]	individual	2016	lapsed 2024	camera/ radio/ sonar/IR	path planning	spoken directions	generic; no motion compensation
US 9,846,231 [32]	STMicroelectronics	2017	see record	ultrasound + accelerometer	sensing gated by gait phase; ToF distance	alarm by distance range	gating idea relevant; different sensor
US 9,993,384 [33]	Toyota	2018	active (per record)	heading sensors	two simultaneous audio cues	stereo audio	navigation audio design; no vision
US 10,299,982 [34]	individual	2019	expired 2023	phone camera + orientation	3-D room model, touch exploration	speech on touch	static room model, not live walking
US 10,809,079 [35]	Ford	2020	active (per record)	GPS, cameras, ultrasonic, mic on vest	static/ dynamic obstacles for routing	audio + haptic	closest in scope; custom garment
US 11,112,261 [9]	Brown University	2021	active (per record)	multiple cameras + inertial	visual localisation	haptic belt/vest	localisation, not obstacle events
US 11,318,050 [8]	American Printing House for the Blind	2022	active (per record)	phone location sensors, beacons	direction-filtered narration of POIs	audio, haptic, braille	navigation narration; no camera

* Legal status as shown on Google Patents on 2026-11-03; Google notes this is an assumption, not a legal conclusion.

4.3 Observations from the patent survey

- **Two clusters.** Patents either sense obstacles (ultrasound, cameras) or support navigation (location narration, haptic direction), with the Ford vest [35] as the main example combining both, on a dedicated garment.

- **Several early patents have lapsed** [29], [31], [34], and the camera + speech concept itself appears in an abandoned 2005 application [30]; the general idea of "camera, recognise, speak" is therefore not novel and cannot be claimed.
- **What the sampled patents do not describe** is the specific combination pursued in NeuroSonic: an unmodified phone browser as a thin client; camera-motion-compensated tracking that announces only objects moving *relative to the background* into or across a walking-path region; suppression of announcements during strong camera motion; and offline speech, commands and routing on a nearby computer. Whether this combination is patentable can only be judged after a full search; this is the subject of a separate invention disclosure.

Chapter 5: Survey: Comparative Analysis of Company Products

Products were chosen to represent the four commercial categories: smart canes, wearable cameras, smartphone apps and robotic or harness guides. Prices are quoted only where an established retailer or a consumer organisation published them, and they change frequently; they are indicative.

5.1 Smart canes

WeWALK is a smart handle that attaches to a standard white cane. It detects obstacles above chest level with an ultrasonic sensor and warns with vibration, connects to a smartphone, and has a touch pad for features such as public-transport information; battery life is quoted as up to five hours, and AARP lists it at US$599 [36], [37]. **SmartCane**, developed at IIT Delhi's Assistech laboratory with the Saksham Trust and manufactured by Phoenix Medical Systems, attaches to a white cane and uses ultrasonic sensing to detect obstacles above knee height, signalling distance with vibration patterns; it has long-range (about 3 m) and short-range (about 1.8 m) modes, and its translational development was funded by the Wellcome Trust [38]. Smart canes extend the cane's range upwards, but they measure distance only; they do not recognise objects or describe motion.

5.2 Wearable cameras

OrCam MyEye is a small camera that clips magnetically onto eyeglasses and works offline, without a smartphone or Wi-Fi; it reads text and, in the Pro version, recognises faces, colours and bank notes, controlled by gestures and voice commands. AARP lists the MyEye Pro at US$4,250; RNIB lists the Pro at £3,700 and the cheaper Smart version at £2,700 (both ex VAT) [36], [39]. It is a reading and recognition aid rather than a mobility aid.

5.3 Smartphone apps

Microsoft Seeing AI is free, with no subscriptions or in-app purchases, and organises tasks into "channels" (reading text, identifying products, recognising currency) [12]. **Google Lookout** (Android) offers several modes, including an Explore mode that identifies objects on the device, plus text and product recognition [40]. **Be My Eyes** connects a blind user by live video to a sighted volunteer, and its *Be My AI* feature describes a photo and lets the user ask questions about it [12]. These apps are free or low-cost and use a phone the user already has, but they work on single photos or on demand, need the user to point the phone, and several of their richer functions need the internet.

5.4 Harness and robotic guides

biped.ai NOA is a shoulder-worn harness with depth cameras covering a wide field of view (a 170° field is reported) that predicts object trajectories a few seconds ahead and warns with 3-D sound; a RealSense case study states that it uses the D430 depth module [41]. **Glidance Glide** is a two-wheeled, self-guided mobility aid pushed by the user, which uses cameras and sensors to steer around hazards; prices reported in early 2025 were US$1,500 plus a monthly subscription, and it was still in development and testing at the time of the reports [42].

Table 4. *Comparative analysis of company products (indicative; checked 2026-11-03)*

Product	Category	Sensing	Recognition	Motion / path events	Navigation	Offline	Indicative price
WeWALK [36], [37]	smart cane handle	ultrasonic	no	no	via phone app	obstacle sensing yes	US$599

Product	Category	Sensing	Recognition	Motion / path events	Navigation	Offline	Indicative price
SmartCane [38]	smart cane add-on	ultrasonic	no	no	no	yes	not verified
OrCam MyEye Pro [36], [39]	wearable camera	camera	text, faces, colours, notes	no	no	yes	US$4,250
Seeing AI [12]	phone app	phone camera	text, products, currency, scenes	no	no	partly	free
Google Lookout [40]	phone app (Android)	phone camera	objects, text, products	no	no	partly (Explore on device)	free
Be My Eyes / Be My AI [12]	phone app	phone camera	human volunteer; AI photo description	no	no	no	free
biped NOA [41]	shoulder harness	depth cameras	obstacles	trajectory prediction	GPS guidance (reported)	see vendor	not verified
Glidance Glide [42]	wheeled robot guide	cameras, sensors	hazards	steers around	yes	see vendor	US$1,500 + subscription (2025 reports)
NeuroSonic	phone + laptop prototype	phone camera, mic, GPS	23 obstacle classes	entering / crossing events	offline map, A* routing	yes (all local functions)	existing phone + laptop

5.5 Observations from the product survey

- **A price gap.** Dedicated wearables cost from about US$600 (cane handle) to over US$4,000 (wearable camera), whereas the phone apps are free. A system that uses hardware people already own addresses the affordability issue raised by the statistics.
- **A capability gap in the free apps.** The free apps recognise and describe, but on demand; they do not continuously warn about objects moving into the walking path. Continuous, motion-aware warning is offered only by dedicated hardware such as NOA.
- **Connectivity.** Richer descriptions depend on the internet; mobility support should not, because connectivity is unreliable exactly where people walk.

Chapter 6: Technology Trend: Introduction and Adaptation Status

Fig. 4 places the surveyed documents on a timeline. Five technology trends run through them. For each we describe the technology, its state of adoption in assistive products, and how it is adapted in this project.

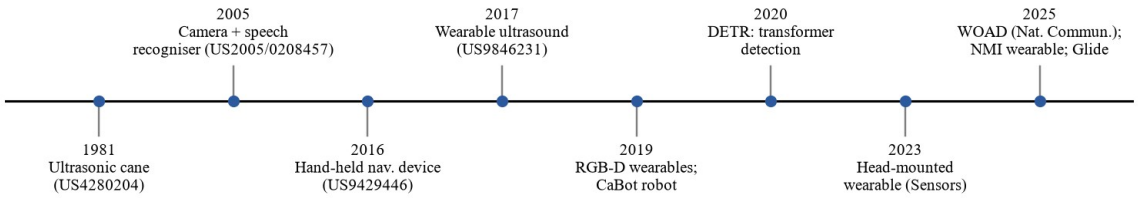


Fig. 4. *Timeline of the surveyed patents, publications and products (positions evenly spaced; years from the cited documents): from ultrasonic canes to camera-based, learning-based wearables.*

Description. The timeline places the surveyed publications, patents and products in order of year. It shows a clear drift from ultrasonic canes towards camera-based, learning-based wearables, and it marks where this project's phone-based approach fits in that development.

6.1 From range sensing to recognition

Early ETAs measured distance with ultrasound [29], [32], and ultrasonic smart canes are still the most widely available products [37], [38]. Since about 2016, deep convolutional detectors such as YOLO [4], SSD [10] and mobile networks such as MobileNetV3 [11] made it feasible to name objects in real time. Products like Seeing AI and Lookout show that recognition is now *commercially mature* for on-demand use [12], [40]. *Adaptation here:* NeuroSonic detects 23 obstacle-relevant classes from the COCO dataset [43] in every frame, rather than on demand.

6.2 Transformer-based detection

DETR [5] reformulated detection as set prediction with transformers [20], removing the hand-tuned non-maximum suppression step. Real-time variants now exist; RF-DETR [44] builds on the self-supervised DINOv2 backbone [45] and is released under the permissive Apache-2.0 licence. The base paper also uses transformer blocks, but for *fusion* of two modalities inside a YOLOv5 detector [3]. *Status:* research and open-source tools are mature; we did not find a surveyed assistive product that states it uses a detection transformer. *Adaptation here:* four detectors were benchmarked on the same images, and RF-DETR Nano was selected for its accuracy, recall and licence (see the implementation and results).

6.3 Multi-modal and depth sensing

Research wearables overwhelmingly combine RGB or infrared with depth [3], [21], [22], [7], and the NOA harness brings depth cameras to a product [41]. Depth gives distance directly, which simplifies warnings. *Status:* established in research, early in products, and expensive. *Adaptation here:* the phone camera has no depth stream that a browser can use, so NeuroSonic deliberately makes no distance statements and uses image-space motion instead; monocular depth estimation is listed as future work.

6.4 Edge, offloading and dedicated hardware

Systems differ in where they compute: on an FPGA plus phone [3], on a separate processor [22], fully on the wearable [7], or in the cloud [12]. Offloading to a nearby, more powerful computer over a local network is a middle path: lower latency and better privacy than the cloud, more capacity than a phone. *Status:* standard in robotics and gaming, uncommon in assistive products. *Adaptation here:* the iPhone is a thin browser client; a laptop GPU on the same Wi-Fi performs detection, tracking, speech recognition, speech synthesis and routing, with no internet required.

6.5 Vision-language models and offline speech

Large multimodal models now describe whole scenes in natural language and answer questions about a photo, as in Be My AI [12]; APIs such as Gemini [46] make this available to developers. They depend on the internet and can state things with unjustified confidence. In parallel, offline speech technology has improved: Kaldi-based Vosk recognises speech locally [47], and Piper [48], built on VITS [49], synthesises natural speech on a CPU. *Status:* cloud description is commercially deployed; high-quality offline speech is mature in open source. *Adaptation here:* local speech in and out is the default; cloud scene description is optional, opt-in, rate limited and filtered so that it never states distance or safety.

Table 5. *Technology trends and adaptation status*

Trend	Maturity in products	Adapted in NeuroSonic
Ultrasonic range sensing	mature (smart canes)	no (cane remains the user's primary aid)
CNN object recognition	mature (apps, on demand)	yes, continuous, 23 classes
Detection transformers	open source; not seen in surveyed products	yes (RF-DETR Nano, FP16)
Depth / multi-modal fusion	research; early products	no (base paper only); future work
Multi-object tracking	robotics; trajectory prediction in NOA	yes, with camera-motion compensation
Local offloading (LAN)	uncommon	yes (phone → laptop GPU)
Vision-language description	deployed (cloud)	optional, opt-in
Offline neural speech in/out	open source	yes (Piper, Vosk)
Offline maps and routing	common in navigation apps	yes (OpenStreetMap, A*)

Chapter 7: Synthesis and Conclusion of the Survey

The survey answers three questions that shape the rest of the project.

Is the problem significant? Yes. At least 2.2 billion people have a vision impairment, 43.3 million were blind in 2020 and 61 million are projected by 2050, and in India about 2 % of people aged 50 and over are blind, rising to 11.6 % above 80 [13], [15], [17]. The population is large, ageing and largely in settings where cost matters.

What do existing solutions leave open? Research systems achieve strong results with dedicated hardware: depth glasses, FPGAs, haptic skins and robots [3], [7], [26]. Products split into affordable but limited smart canes, expensive wearables, and free phone apps that work on demand rather than continuously [36], [12]. Patents cover range sensing, camera recognition with speech, and navigation narration, but the sampled patents do not describe continuous, camera-motion-compensated warnings of objects moving into the walking path from an unmodified phone [35], [30], [8].

Which technologies are ready to use? Real-time detection transformers with permissive licences, standard multi-object tracking, offline neural speech recognition and synthesis, and open map data are all mature [44], [28], [47], [48], [50]. Depth on ordinary phones and dependable cloud description are not yet ready for a mobility aid.

Direction for the project. Re-implement and verify the base paper (see the base-paper re-implementation); then build a system that uses an ordinary phone as camera, microphone, GPS and speaker; runs perception, speech and routing offline on a nearby laptop GPU; warns about objects moving into or across the walking path while ignoring the user's own movement; and states honestly what it can and cannot see (see the implementation and results). The system is a supplement to the white cane, not a safety device.

The main limitation of this survey is its breadth: it is a structured sample rather than a systematic review, the patent search was limited to US documents on Google Patents, and product details come from manufacturer-adjacent or retailer sources that change over time. These limits are stated so that the comparisons are read as indicative.

Chapter 8: The Base Paper: WOAD

8.1 System overview

WOAD addresses a stated tension: wearable obstacle-avoidance devices find it hard to be reliable, responsive, long-lasting and usable at the same time [3]. Its answer has four parts.

- **Sensing glasses.** Self-developed glasses of about 400 g, including an 80 g battery, carry a Synexens CS30 RGB-ToF module. Its near-infrared (NIR) camera and ToF depth sensor share a 100°×75° field of view at 640×480 pixels; depth is measured from 0.1 to 5 m.
- **Depth-aided compression.** To reduce transmission delay to the phone, a reinforcement-learning agent (discrete soft actor-critic [18]) uses obstacle status from consecutive depth frames to decide whether to send a frame and at what quality. The authors report a compression ratio above 95 %.
- **FPGA.** The compression policy runs on a custom 28 nm board with a multi-float-point vector streaming architecture; the authors report a latency of 0.095 ms and over 50 % lower power than their reference.
- **Cross-modal detector on the phone.** A Samsung Galaxy S23 Ultra runs a TensorFlow Lite version of a YOLOv5-based detector [19] with two input streams (NIR and colour-mapped depth) fused by transformer blocks [20]. Alerts are spoken at three hazard levels (attention, warning, danger), and three vibration motors on the forehead and temples indicate direction; the vibration frequency encodes distance (200 Hz below 3 m, 150 Hz above).

8.2 Detector architecture

Fig. 5 shows the detector as we verified it on the released checkpoint (run). The NIR image passes through the`exp72` first three YOLOv5s layers and the depth image through a separate stem of the same structure, each producing a 64×160×160 feature map. Both are scaled by a fixed 0.5 gate, average-pooled to 8×8, and flattened into 128 tokens of 64 dimensions with a learned positional embedding. Eight transformer blocks with eight attention heads let every NIR token attend to every depth token. The output tokens are reshaped, upsampled, and used to re-weight each stream's features; the streams are concatenated, fused by a convolution, and passed through the standard YOLOv5s backbone, PAN neck and detection head, which outputs 25,200 candidate boxes for three classes (person, Ebike, Car). The checkpoint has 7,521,992 parameters (the paper reports 7,497,914 for its model [3]; the small difference may come from a different training run or from counting, and does not affect the analysis).

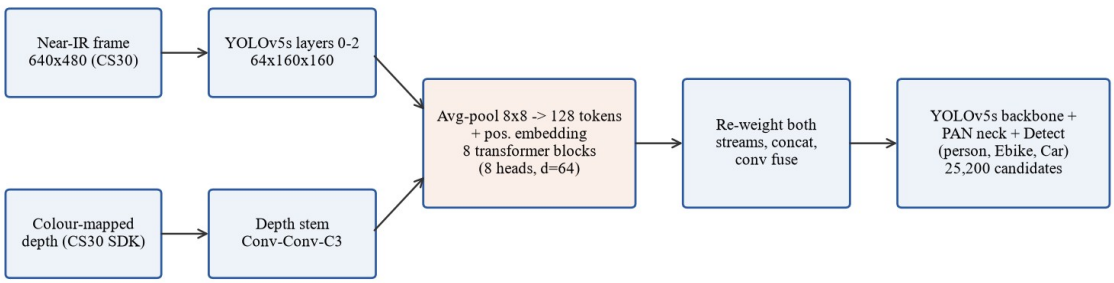


Fig. 5. *WOAD cross-modal detector as verified on the released exp72 checkpoint: two input streams (near-infrared and colour-mapped depth), transformer fusion of 128 tokens, and a YOLOv5s detection network.*

Description. The block diagram shows the architecture of the base paper's WOAD detector. A near-infrared stream and a colour-mapped depth stream are fused by a transformer of 128 tokens and decoded by a YOLOv5s head. Because it needs both inputs from a special sensor, it cannot run on an ordinary phone camera.

8.3 Results reported by the authors

Table 6. Results reported in the base paper [3] (not measured by us)

Measure	Reported value
Collision avoidance, all scenarios	100 %
White cane, sudden indoor obstacles / fast cars at night	≈ 26 % / ≈ 40 %
End-to-end delay	< 320 ms
Operating duration; power	≈ 11 h; ≤ 4 W
System Usability Scale (WOAD vs cane)	83.75 ± 7.79 vs 80 ± 22.36
Participants	20 (12 quantitative over seven months, 8 survey)
Detector mAP@0.5 (training plot of the released run)	0.669 (person 0.771, Ebike 0.839, Car 0.395)

The authors also note limitations: occasional false alerts from shadows or snowflakes, concern that the glasses attract attention, and the need for a training period [3]. The detector accuracy in the last row of Table 6 is not stated in the main text of the paper; it is read from the precision–recall plot saved with the released training run, and it is the figure we attempted to reproduce.

Chapter 9: Re-implementation of the Base Paper

9.1 What was released, and its state

The public repository contains five parts: data collection with the CS30 SDK, the reinforcement-learning compression, the FPGA design, the smartphone deployment, and the cross-modal detector with its training run . Code and data`exp72` are distributed through nine Google Drive links. We inspected the detector code, the checkpoint and the published dataset in detail and, for the other links, read the archive indexes without downloading the whole archives.

The detector scripts did not run with current libraries. We found:

- the training, validation and detection scripts call the model with one concatenated input, whereas its forward function expects two separate tensors (NIR and depth);
- use of NumPy aliases that have been removed (`np.int`) and of PyTorch loading defaults that have changed;
- a missing argument in one code path, and a detection script that exits before non-maximum suppression;`clouddir`
- no saved results file and an empty TensorBoard log for the reported run; only the training plots remain.

Modifying the original repository would have made comparison impossible, so it was left untouched and a separate, standalone inference package was written.

9.2 Faithful re-implementation

The package ( ) has three parts. A *loader* reads the pickled checkpoint by temporarily mapping the original`woad/` module names to vendored copies of the nine classes the checkpoint needs. It checks the file's SHA-256 hash and the module tree, and it refuses a second, incompatible five-class checkpoint found at the repository root. A *preprocessing* module reproduces the original validation pipeline: the NIR image is resized so that its long side is 640 pixels (area interpolation) and letter-boxed to 640×640; the depth image is median-filtered (3×3) and letter-boxed; both are converted to RGB channel-first tensors in [0, 1]. The *detector* applies the original non-maximum suppression and coordinate scaling, so its detections are in the original image coordinates.

Equivalence test. A tool runs the original model code in a separate process and our re-implementation on the same seeded inputs and compares the raw outputs. The maximum absolute difference is **0.0**: the re-implementation is bit-identical to the original.

Modality ablation. To confirm that both inputs matter, we measured the total detection confidence when inputs are altered. Swapping NIR and depth leaves 8 % of the confidence; blanking the NIR input removes all detections; blanking the depth input leaves 65 %. Both modalities are used, and NIR dominates. Automated tests encode these properties, and deliberately breaking the preprocessing makes them fail.

9.3 Dataset audit

The detector's published data (`VOCdetiv_data.rar`, 1.31 GB) was downloaded, its size checked against the server and its RAR checksums verified. It holds four nested variants with 4 to 10 classes; the largest (nc_10) has 1,286 training and 365 validation image pairs with classes such as person, box,

bike, zebra crossing, telegraph pole, doors, building entrance and traffic signs. The audit found that **this is not the dataset behind the reported run**:

- the checkpoint's classes are person, Ebike and Car; the dataset has no Ebike or Car class;
- none of the run's 187 validation frames (recovered from the run's plots) appears among the dataset's 1,651 unique images: the best normalised-correlation similarity was 0.72, while the same matcher recovers deliberately degraded dataset frames at above 0.9;
- dates and file paths in the archive differ from those of the run.

We also checked the other Drive links. The code bundle, two unlabelled recordings and the SDK do not contain the labelled person/Ebike/Car data either; one link returns HTTP 404. Other data issues: one orphan label, 53 file names reused between training and validation with different pixels, and one class ("Down stairs") without instances.

9.4 Evaluation method

The only class shared by model and data is *person*, so the evaluation is a cross-dataset person measurement. The evaluation tool reproduces the original validation settings exactly (Table 7). Its AP code was verified to give identical values to the original code on random data.

Table 7. *Evaluation settings (identical to the original validation code)*

Item	Setting
Inputs	NIR (`images/`) + depth (`clouds/`), same file name
Input size	640, letter-boxed
NMS	confidence 0.001, IoU 0.6, max. 300 detections
Matching	IoU 0.50:0.05:0.95
AP	101-point interpolation (a perfect detector scores 0.995)
Classes	dataset 0 (person) → model person; other classes ignored
Precision	FP32 on CPU (original: FP16 on CUDA)

9.5 Results

Table 8. *Measured person accuracy of the WOAD exp72 checkpoint on the published data (nc_10)*

Split	Images	People	P	R	AP@0.5	AP@.5:.95
val	365	129	0.345	0.310	**0.218**	0.073
train	1,286	458	0.361	0.199	**0.173**	0.050

Table 8 gives the results; precision (P) and recall (R) are at the maximum-F1 operating point (confidence 0.18 on val, 0.33 on train). Fig. 6 shows the precision–recall curves. At the original plotting settings (confidence 0.25, IoU 0.45) the model detected 37 of 129 people in the validation split and missed 89, with 5 unmatched detections; in the training split it detected 104 and missed 345. The forward pass took about 126–131 ms per image on the CPU in batches of 8.

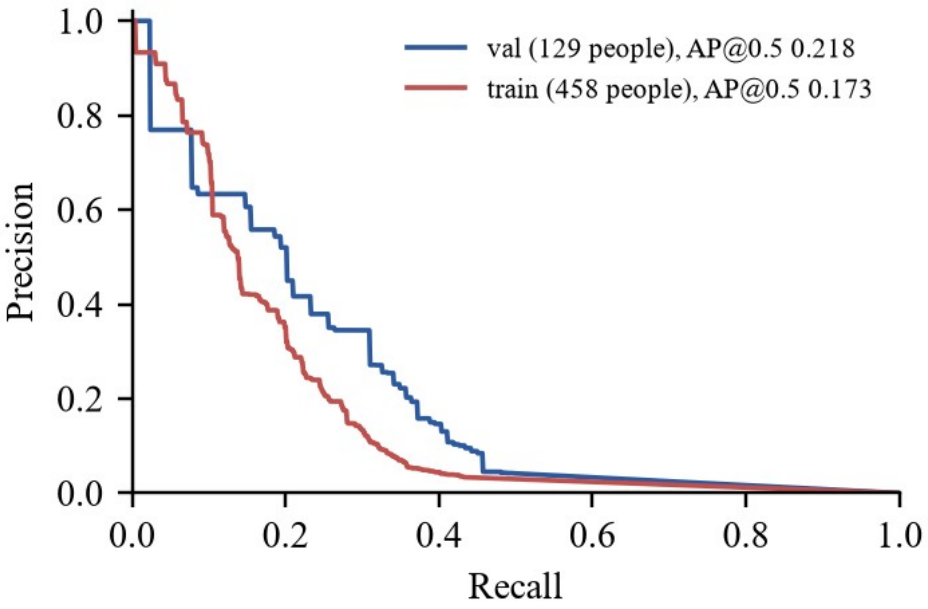


Fig. 6. *Person precision–recall curves (IoU 0.5) of the WOAD checkpoint on the published dataset, measured with the original metric code.*

Description. The curves plot precision against recall for the person class at IoU 0.5 when the WOAD checkpoint is run on the published dataset with the original metric code. The curve falls quickly as recall rises, which is the behaviour behind the low cross-dataset average precision reported in Table 8.

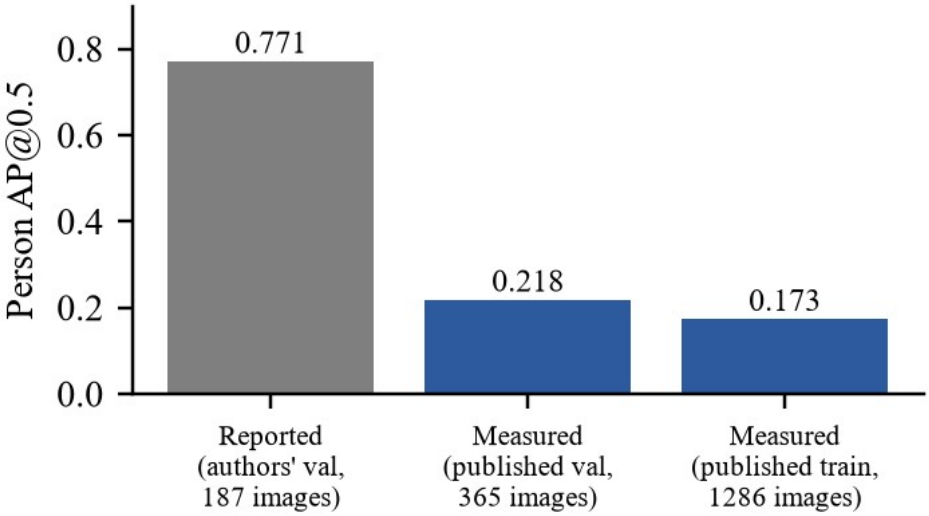


Fig. 7. *Person AP@0.5: value read from the authors' training plot (their own validation split) versus our cross-dataset measurements. The datasets differ, so this is not a like-for-like comparison; it shows that the reported value could not be reproduced with the published data.*

Description. The bar chart sets the person AP@0.5 read from the authors' own training plot beside our two cross-dataset measurements. The large gap shows that the reported value could not be reproduced with the published data; because the datasets differ, the comparison is indicative and not like-for-like.

Interpretation. Fig. 7 contrasts the reported person AP (0.771) with our measurement (0.218 val, 0.173 train). Because the datasets differ, the gap measures *generalisation* to new scenes, not an error in the paper. The model is fairly precise but misses most people in these scenes. Visual inspection of sampled frames showed that close, upright people are detected with high confidence (0.74–0.87),

whereas cyclists and distant people indoors are usually missed. The bicycle class of the dataset scored essentially zero as "Ebike", which is expected given the different class definitions and is not reported as an accuracy figure.

9.6 Brightness and live-input experiment

A raw CS30 recording released with the repository is much darker than the training-like data (mean NIR brightness 59 versus about 140). Darkening training-like frames to that level removed about 73 % of detections, and histogram correction of the dark recording produced one correct detection and 56 false alarms. The model is therefore sensitive to sensor exposure, which motivated an input-quality gate in our implementation that flags frames that are too dark, too low in contrast, or almost without valid depth, and withholds alerts from unusable frames.

9.7 Re-use of the base model in this project

The re-implemented detector is kept in the project as a research path: a command-line tool and a "replay" mode run it on recorded NIR + depth sequences, with an alert engine that requires confirmation over several frames, applies cooldowns and states the object position (left, ahead, right). Thirty-eight original-equivalence and model tests remain part of the automated suite.

Chapter 10: Reproduction Procedure and Repository Audit

10.1 Components of the released work

Table 9. Components of the base paper's public release and their state when received

Component	Content	State when received	Used here
Cross-modal detector	YOLOv5s-based two-stream network with transformer fusion; run exp72; classes person, Ebike, Car	checkpoint present; train/val/detect scripts broken with current libraries	yes: re-implemented for inference
Reported accuracy	mAP@0.5 0.669 in a training plot	no results file; empty TensorBoard log	attempted reproduction
Published dataset	`VOCdetiv_data.rar`, NIR + depth, nc_4 to nc_10	intact, but a different dataset from exp72	yes: cross-dataset evaluation
Data collection	CS30 SDK binaries, capture program	no capture source code	only the colour-map finding
RL compression	D-SAC training scripts	data link returns HTTP 404	no
FPGA	Vivado project for the policy network	not inspected in depth	no
Smartphone	Android application	inspected by archive index; per the project's audit it ships a stock COCO TFLite model	no

10.2 Procedure

1. **Isolate the original.** The original repository was copied read-only; no file in it was changed, so every comparison is against the authors' code as published.
2. **Identify the checkpoint.** The exp72 checkpoint was hashed (SHA-256 prefix `da043ff2`), its module tree listed and its dimensions read: eight transformer blocks, eight heads, model width 64, positional embedding 1×128×64, 7,521,992 parameters.
3. **Re-implement inference.** Loader, preprocessing, non-maximum suppression and coordinate scaling were written as a standalone package with the original numerical behaviour.
4. **Prove equivalence.** The original model code and the re-implementation were run in separate processes on identical seeded inputs; the outputs were identical.
5. **Test that the inputs matter.** Input swapping and blanking (ablation) and deliberate preprocessing faults were added as automated tests.
6. **Audit the data.** The published archive was downloaded and verified; classes, counts, duplicates and frame identity against the run's validation set were checked; the other download links were inspected through their archive indexes.
7. **Evaluate.** Person AP was computed with the original matching and AP code, verified to be numerically identical to the original on random data.
8. **Probe the live-input path.** A raw sensor recording was compared with the training-like data, and the effect of brightness on detections was measured.

10.3 Download links

Nine Google Drive links are listed in the repository. File names, sizes and dates were read with HTTP HEAD requests, and archive contents from each RAR5 archive's index with a few small HTTP range

requests (validated by listing the downloaded dataset and comparing all 9,370 file sizes). The code bundle (1.70 GB) contains the YOLOv5 code, a third checkpoint of a different size, and the same images with labels only inside training caches. Two archives hold unlabelled NIR + depth sequences (600 and 501 frames), which are useful for qualitative replay. One archive holds the SDK binaries, others hold Android build files, the RL project and the FPGA files. One link is dead (HTTP 404). No link contains the labelled person/Ebike/Car data of the reported run.

Chapter 11: Threats to Validity of the Re-implementation

- **Numerical precision.** We evaluated in FP32 on the CPU; the original validation used FP16 on CUDA. The effect is expected to be negligible but was not measured.
- **Different data.** The measured AP is cross-dataset. It shows weak generalisation to the published scenes, not an error in the authors' own validation. Whether some published images were in the run's *training* data cannot be checked; the low scores make large overlap unlikely.
- **Metric quirks of the original code.** A perfect detector scores 0.995, not 1.0, with the original AP interpolation; and in the original confusion analysis unmatched detections are counted only in images that also contain a match, so false positives are under-stated. We kept these behaviours deliberately, to stay comparable.
- **Class mapping.** Only person is semantically shared. Bicycles scored as "Ebike" (AP ≈ 0) are reported only as an exploratory observation.
- **Label anomalies.** One orphan label and 53 file names reused across splits with different pixels; no image was excluded, following the original label rules.

Chapter 12: Building on the Base Model: Alert Engine and Input-Quality Gate

To turn per-frame detections of the base model into spoken alerts for recorded sensor data, an alert engine and an input-quality gate were implemented around it. They are retained in the final system as its research "replay" path.

Table 10. *Alert-engine and quality-gate parameters (defaults)*

Parameter	Value
Alert confidence threshold	0.40
Confirmation: hits within window	2 of 3 frames
Track association IoU	0.30
Cool-down / repeat interval	5 s / 15 s
Max. object alerts per frame	2
Position bands (left / ahead / right)	<1/3, 1/3–2/3, >2/3
Class priority	Car > E-bike > person
Quality "warn": NIR mean / std below	100 / 40
Quality "unusable": NIR mean / std below	40 / 10
Quality "unusable": invalid depth above	95 %

The engine confirms an object when it is seen in two of three consecutive frames, groups objects of the same class and position into one message (e.g. "Person detected ahead."), orders messages by class priority and box size, applies cool-downs, and announces when detection becomes unavailable or resumes. The quality gate compares each frame's NIR brightness and contrast and the fraction of invalid depth pixels with thresholds derived from the brightness experiment, and it withholds object alerts from "unusable" frames instead of speaking possibly meaningless detections. Both components are covered by automated tests. Their design principles (temporal confirmation, position words instead of distances, explicit "unavailable" messages) carried over into the phone-camera system.

Chapter 13: Cons of the Base Work and Gap Identification

The strengths of WOAD are real: an integrated hardware–software system, a principled two-modality detector, energy-aware compression, and an unusually thorough user study [3]. The re-implementation also exposed weaknesses. We separate *cons* (properties of the base work) from *gaps* (what is missing for the target use: an ordinary phone, Indian streets, low cost, offline operation) and map each gap to a contribution.

13.1 Cons observed

1. **Hardware lock-in.** The detector requires NIR images from the CS30 sensor and depth colour-mapped by the vendor SDK's own look-up table (not a standard colour map, and with undocumented range settings). An ordinary RGB camera cannot drive it: feeding RGB as "NIR" with a fake depth image is out of distribution, and our ablation shows that NIR carries most of the signal.
2. **Reproducibility.** The reported detector accuracy cannot be re-measured with the published data, the scripts did not run with current libraries, and the run's results files are missing.
3. **Generalisation.** On the published scenes, person AP@0.5 is 0.17–0.22; cyclists and distant people are mostly missed. For a warning system, low recall on people is a safety-relevant weakness.
4. **Exposure sensitivity.** Darker sensor recordings remove most detections.
5. **Narrow class set.** Three classes (person, Ebike, Car) in the released model.
6. **No distance in the released detector code.** The paper's device encodes distance in vibration, but the released detector outputs only class, box and confidence; distance would need the raw 16-bit depth, which the colourised image does not preserve.
7. **Frame-by-frame semantics.** The released detector code has no tracking, so it cannot distinguish an object moving into the path from the apparent motion caused by the wearer's own walking.
8. **Obstacle-only.** There is no route guidance or speech-command interface in the released code.

Table 11. *Gap identification and the contribution that addresses each gap*

#	Gap	Evidence	Consequence	Proposed contribution
G1	Needs a specific NIR + ToF sensor and custom glasses	model inputs; ablation (blank NIR → 0 detections)	unusable with a phone; costly	phone RGB camera as input; detector chosen for RGB
G2	Reported accuracy not reproducible	dataset audit; missing run files	performance unknown	controlled benchmark of 4 detectors on the same annotated images
G3	Low person recall on new scenes	AP@0.5 0.218 / 0.173	missed warnings	select by recall; temporal confirmation absorbs false positives
G4	No motion semantics	frame-wise code	cannot warn "moving into path" reliably	camera-motion-compensated tracking; entering and crossing events
G5	No distance in released code; risk of over-claiming	outputs: class, box, score	misleading messages possible	no distance or "safe" statements; walking-path states only

#	Gap	Evidence	Consequence	Proposed contribution
G6	No navigation or voice interface	released code	user needs other apps	offline map, A* routing, spoken turns, offline voice commands
G7	Dependence on custom FPGA and compression for latency	system design	hard to replicate	off-the-shelf laptop GPU over local Wi-Fi; latency measured at the real frame rate
G8	Connectivity and privacy	cloud alternatives need internet	unreliable outdoors	all local functions offline; images never stored; cloud description opt-in

Gaps G1 and G2 are decisive: they mean that the base model cannot be the live detector of a phone-based system. The proposed work therefore keeps WOAD as a verified research baseline and builds a new perception path for the phone camera, while addressing G3–G8 in the system design.

Chapter 14: Proposed Work

14.1 Problem statement

Provide a pedestrian who carries an ordinary smartphone with short, timely spoken information about recognised objects in the walking path, including objects moving into or across it, together with spoken route guidance; without depending on the internet, without storing camera images, without special sensors, and without overstating what the system can see.

14.2 Objectives

1. **O1 (real time):** process phone frames at about 5 frames/s, with an answer well within one frame interval (200 ms).
2. **O2 (detector):** choose the detector by measured accuracy, recall, latency and licence, with a CPU fall-back.
3. **O3 (motion):** announce people or vehicles *entering* or *crossing* the walking path, but not the apparent motion caused by the user's own walking.
4. **O4 (offline):** keep detection, speech output, voice commands and routing working without internet; make cloud description opt-in.
5. **O5 (honesty):** never state distances or safety; say "appears clear", never "safe".
6. **O6 (verification):** verify each function with automated tests, and state what is not verified.

14.3 Proposed architecture

The phone becomes a thin client running in Safari. It captures camera frames, microphone audio and GPS, and plays speech. A laptop with a consumer GPU on the same Wi-Fi network runs all engines (Fig. 8). This replaces the custom glasses, FPGA and compression of the base system (G1, G7) with hardware the user is likely to have or can borrow, at the cost of depending on the laptop.

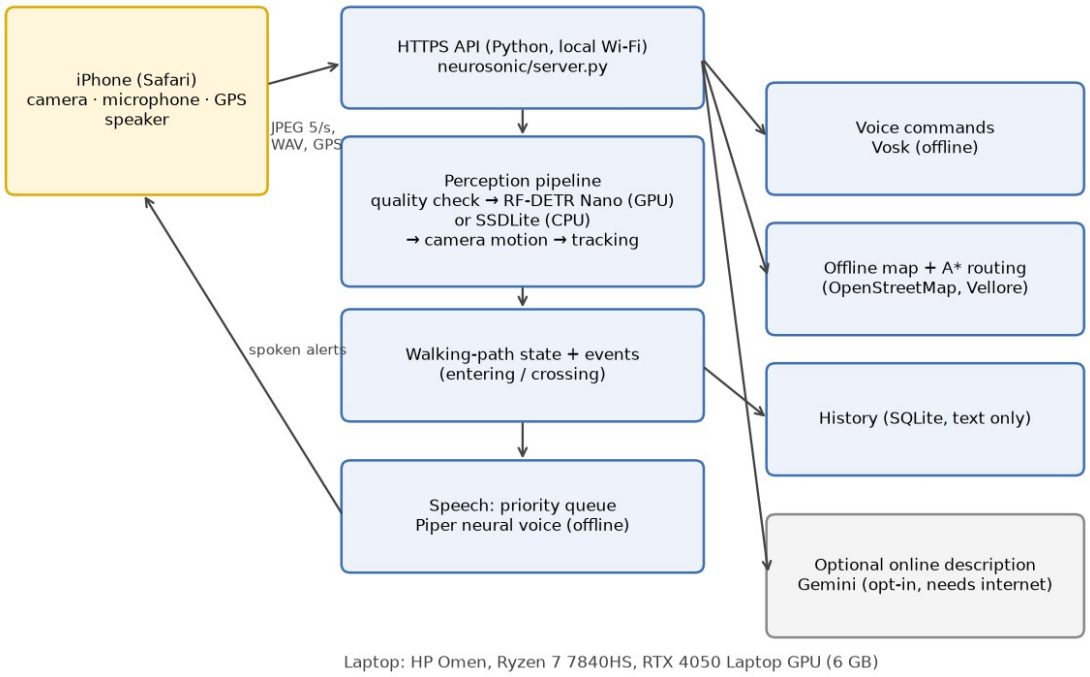


Fig. 8. *Proposed architecture. The phone (browser) sends frames, audio and position over HTTPS on the local network; the laptop runs perception, events, neural speech, offline voice commands, offline map routing and text-only history. Only the optional scene description uses the internet, after the user opts in.*

Description. The diagram shows the proposed system. The iPhone browser captures camera frames, audio and position and sends them over HTTPS on the local network. The laptop runs the detector, tracker, events, Piper speech, Vosk voice commands, offline map routing and text-only history, and only the optional scene description uses the internet.

14.4 Methodology

- **Detector selection (G1–G3).** Benchmark candidate RGB detectors trained on COCO [43], from the established SSDLite/MobileNetV3 [10], [11] to the YOLO family [51] and the real-time detection transformer RF-DETR [44], [5], on the same annotated person images with the same AP code used for WOAD; select by recall first, because a missed person is worse than an extra caution.
- **Camera motion and tracking (G4).** Estimate the camera's own motion between frames from background corner points [52] tracked with pyramidal Lucas–Kanade flow [53], [54] and a RANSAC similarity fit [55]; predict each track's box with that motion and associate detections in two stages as in ByteTrack [28], [27]; store each object's motion *relative to the background*.
- **Walking-path states and events (G4, G5).** Define a lower-central walking-path region; confirm path states over 2–3 frames; raise "entering" and "crossing" events only for confirmed tracks with sufficient background-relative speed, and suppress all events during strong camera motion.
- **Speech, commands and routing (G6, G8).** Offline neural speech with Piper [48], [49], offline command recognition with Vosk [47], and A* routing [56] on an OpenStreetMap extract [50] of Vellore, with spoken turn instructions.
- **Serving at the real frame rate (G7).** Measure latency at 5 frames/s, not only back to back, and correct any serving bottleneck found.

14.5 Evaluation plan

Table 12. Evaluation plan for the proposed work

Question	Measure	Where
Which detector?	person AP@0.5, recall @0.40, GPU/CPU latency, licence	laptop benchmark
Does FP16 change accuracy?	AP and latency FP32 vs FP16	laptop benchmark
Real-time?	frames/s, capture→answer time	browser test, fake camera
Stable over time?	errors, latency, memory over 10 min	soak test
Correct events, no false alarms from panning?	unit tests with synthetic camera motion	automated
Offline?	functions with network blocked	automated, browser
Works on iPhone, Wi-Fi, streets, users?	23-scenario protocol, 5 trials each	field (pending)

14.6 Differentiation from the base paper

Table 13. Base paper versus proposed work

Aspect	WOAD (base)	Proposed (NeuroSonic)
Sensor	NIR + ToF glasses (CS30)	phone RGB camera
Compute	FPGA + phone	laptop GPU over local Wi-Fi
Detector	YOLOv5 + transformer fusion, 3 classes	RF-DETR Nano, 23 classes, CPU fall-back
Temporal logic	per frame	motion-compensated tracking, events
Distance	from depth (device)	none stated (honest by design)
Navigation	none	offline map, A*, spoken turns
Voice input	none	offline commands
Cost to user	custom device	existing phone + laptop
User study	20 participants	not yet (protocol prepared)

Chapter 15: System Architecture

15.1 Deployment

The server is a standard-library Python HTTP(S) server on the laptop. The phone loads a web application (HTML, CSS and JavaScript) from it; nothing is installed on the phone. Safari grants camera and microphone access only to secure pages, so the server uses HTTPS with a certificate issued by a locally generated certificate authority. The user installs the authority once from a plain-HTTP setup page on the next port; browser security is never disabled. The deployment architecture is shown in Fig. 8 and the per-frame pipeline in Fig. 9.

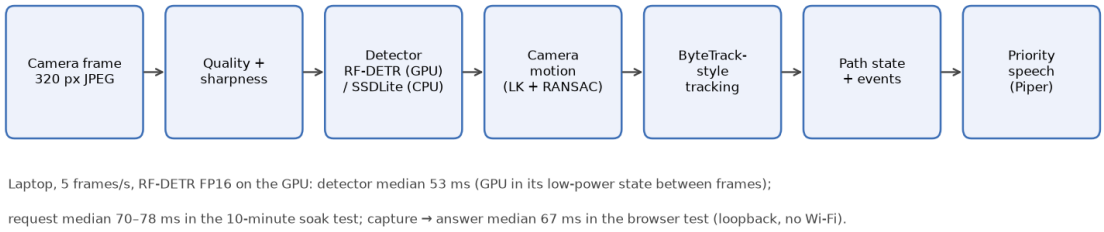


Fig. 9. *Per-frame perception pipeline on the laptop: frame quality check, detector (RF-DETR on the GPU or SSDLite on the CPU), camera-motion estimation, tracking, walking-path state and events, and the phone's priority speech queue.*

Description. The pipeline diagram follows one frame through the laptop. It passes a quality check, then the detector (RF-DETR on the GPU or SSDLite on the CPU), camera-motion estimation, tracking and walking-path state, and ends in the phone's priority speech queue. Each stage is a separate module so it can be tested alone.

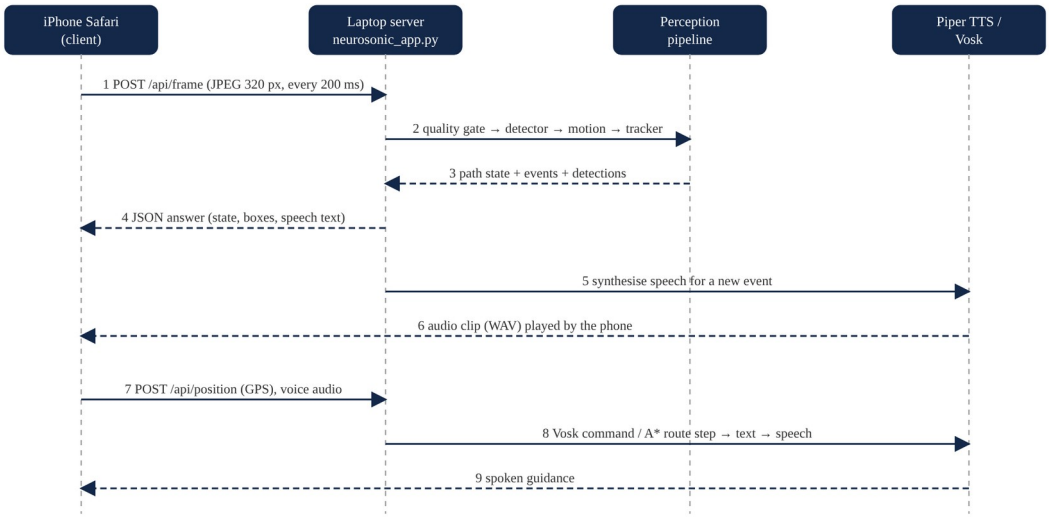


Fig. 10. Message sequence between the iPhone, the laptop server, the perception pipeline and the speech modules.

Description. Steps 1–4 are the frame loop: the phone posts a JPEG, the server runs the pipeline and returns the state and speech text. Steps 5–6 return a neural-voice clip when an event occurs. Steps 7–9 show position updates and voice commands, which are turned into spoken navigation guidance. Only one request is in flight at a time.

15.2 Phone client

- Posts a 320-pixel-wide JPEG every 200 ms (400 ms if the laptop reports the CPU detector), with at most one request in flight, so a slow answer reduces the frame rate instead of building a queue.
- Records speech as WAV for command recognition when the user presses the talk button.
- Uses GPS locally and sends a position only to request a route.
- Plays speech through a priority queue (urgent > system > obstacle > navigation/scene > information) with per-category cool-downs; urgent warnings interrupt routine speech, and an interrupted navigation step is replayed once.
- Shows a detector chip on the home screen (RF-DETR GPU or SSDLite CPU) so that a sighted helper can see which detector is active.

15.3 Laptop server

Table 14. *Main server modules*

Module	Responsibility
`server.py`	HTTP(S) API (16 GET, 12 POST endpoints); per-connection TLS handshakes in worker threads
`detectors.py`	RF-DETR (CUDA, FP16) and SSDLite (CPU) behind one interface with fall-back
`awareness.py`	frame pipeline, single inference worker thread, client statistics
`tracking.py`	camera-motion estimation, motion-compensated two-stage tracker
`events.py`	entering / crossing rules and their wording
`pathguide.py`	walking-path states and their wording
`tts.py`	Piper neural voice (Indian English), cached WAV with HTTP range support
`voicecmd.py`	Vosk two-pass recognition and intent parser
`maps.py`	offline OpenStreetMap graph, place search, A* routes
`store.py`	SQLite history and saved places, text only
`vision/`	optional cloud scene description (Gemini) with consent, rate limit and wording filter

15.4 Privacy and safety by design

Camera images are processed in memory and are never written to disk, logs or the database; the history stores text messages only. The cloud description sends one image to a third party only after an explicit opt-in. Every spoken message is drawn from a fixed vocabulary that never states a distance and never says that a way is safe; the cloud description is filtered for the same words. The application states on its "How it works" screen that it is not a replacement for a cane, guide dog or human assistance.

Chapter 16: Methodology

16.1 Detector selection

Four candidates were run through the same benchmark harness: the previous production detector SSDLite320-MobileNetV3-Large [10], [11] (torchvision), RF-DETR Nano [44] (rfdetr 1.11.2), and YOLO26n and YOLO26s [51] (ultralytics 8.4.198, used for the benchmark only). All are trained on COCO [43].

- **Quality** was measured as person average precision with the same AP code used to evaluate WOAD, on 365 near-infrared validation images containing 129 annotated people. These are the only locally available images with person boxes. They are not phone RGB images, so they *rank* the detectors but do not predict street performance. Precision and recall are also reported at the production confidence threshold of 0.40.
- **Latency** was the mean over 200 (GPU) or 60 (CPU) recorded 640×480 frames, back to back, FP32, batch 1, after 10 warm-up frames.
- **Licence** was a criterion because the system is a network service: AGPL-3.0 would oblige release of the source code to its users.

16.2 Detector runtime and fall-back

RF-DETR runs on CUDA from local weights; downloads are blocked at run time. Detections are restricted to 23 obstacle-relevant COCO classes (e.g. person, bicycle, car, motorcycle, bus, truck, dog, cow, chair, bench, potted plant) at confidence ≥ 0.40. A wrapper falls back to SSDLite on the CPU if CUDA, the package or the weights are missing, if warm-up fails, or if a run-time inference fails, in which case the same frame is re-analysed by SSDLite. Three serving changes were made after measurement: PyTorch is limited to two CPU threads while the GPU detector is active; all inference runs on one long-lived worker thread rather than on the HTTP server's per-request threads; and RF-DETR runs in FP16 without graph compilation, with FP32 as automatic fall-back.

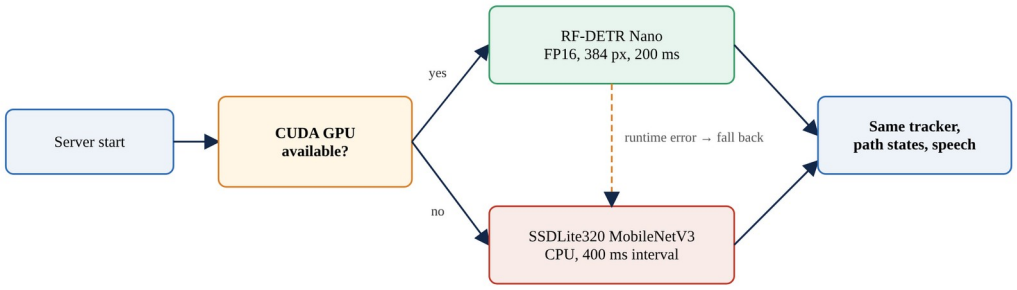


Fig. 11. Detector selection and fall-back at start-up and at run time.

Description. At start-up the server checks for a CUDA GPU. With a GPU it uses RF-DETR Nano in FP16 at a 200 ms interval, without one it uses SSDLite320 on the CPU at 400 ms. A run-time error on the GPU path also falls back to SSDLite. The same tracker, path states and speech are used in both cases.

16.3 Camera motion estimation

Up to 150 Shi–Tomasi corners [52] are taken from a 320-pixel grey image, *outside* the detected boxes so that moving objects do not bias the estimate. They are followed into the next frame with pyramidal Lucas–Kanade optical flow [53], [54], and a similarity transform (translation, scale, rotation) is fitted with RANSAC [55] using OpenCV [57]. With fewer than 12 corners or inliers the camera motion is marked "unknown". The transform M maps a point in the previous frame to where a static point would appear now.

16.4 Motion-compensated tracking

Each track's box is predicted by applying M to its last box. Association follows the two-stage idea of ByteTrack [28], in place of SORT's Kalman prediction [27]:

- detections with confidence ≥ 0.55 are matched first, greedily by IoU, within each class;
- detections between 0.40 and 0.55 may only extend existing tracks;
- tracks end after three missed frames.

Every update stores the object's displacement relative to the background, i.e. the observed displacement minus the displacement predicted from camera motion alone:

$$\Delta = c - M(c)$$

where c is the box centre. A parked car seen while the user turns has $\Delta \approx 0$, whereas a person stepping sideways has a large Δ.

16.5 Walking-path states and events

The walking path is a lower-central trapezoid of the image. Its state is CLEAR_VIEW, POSSIBLE_OBSTACLE, BLOCKED_VIEW, UNCERTAIN (dark or blurred frame) or CAMERA_UNAVAILABLE, confirmed over 2-of-3 or 3-of-3 frames before it is spoken, with chest-mounted and hand-held wording (e.g. "The visible path appears clear. Continue carefully."). Events are considered only for confirmed tracks (at least three updates over at least 0.4 s) of people, vehicles or animals:

- *Entering:* the object was outside the path at the start of a 1.5 s window, overlaps it now, and moves towards it at ≥ 0.12 image widths per second relative to the background.
- *Crossing:* the object's centre passes from below 35 % to above 65 % of the image width (or back) within 3 s at ≥ 0.15 widths/s, with the bottom of its box in the lower half of the image.

Nothing is announced while the camera shifts more than 6 % of the width per frame, zooms more than 4 %, or has no motion estimate; nothing is announced in BLOCKED_VIEW, UNCERTAIN or CAMERA_UNAVAILABLE; each track triggers at most one event; and events are at least 3 s apart. An example message is "Caution. A person is moving into your path from the left." An "approaching" event was deliberately not implemented: with a moving camera, box growth is dominated by the user's own walking.

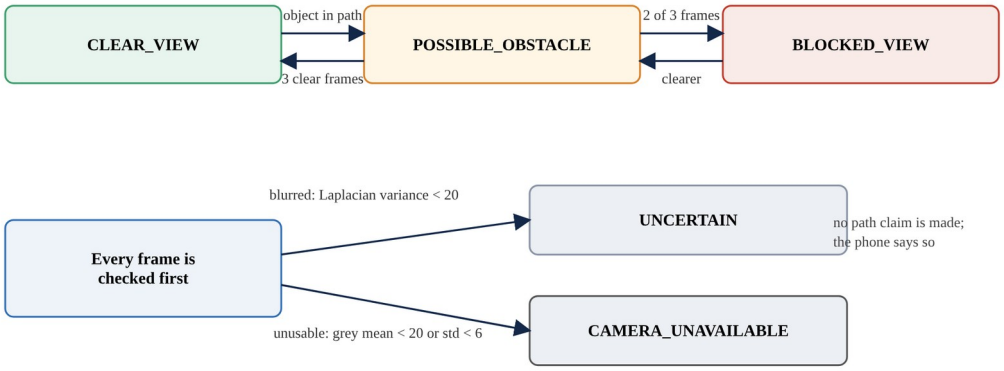


Fig. 12. Walking-path states and the transitions between them.

Description. The state machine has three view states (clear, possible obstacle, blocked) and two guard states. A clear view is declared after three clear frames, a blocked view after two of three obstructed frames, and every frame is first checked for blur (Laplacian variance below 20) and for unusable brightness (grey mean below 20 or standard deviation below 6).

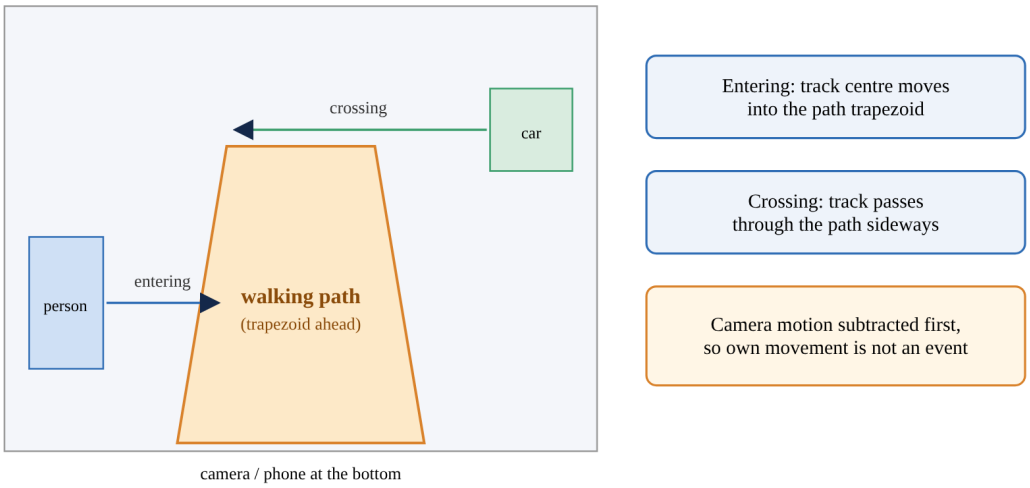


Fig. 13. Walking-path trapezoid and the entering and crossing events.

Description. The shaded trapezoid is the area in front of the user that counts as the walking path. A tracked object whose centre moves into it is an entering event, and one that passes sideways through it is a crossing event. Camera motion is subtracted before this test, so the user's own turning is not announced as an event.

16.6 Speech, voice commands and routing

- **Speech output:** Piper [48] (VITS-based [49]) with the Indian-English voice `en_IN-spicor`; results are cached and delivered as WAV with HTTP range support (needed by iOS); the phone's own voice is the fall-back.
- **Voice commands:** Vosk [47] with the small Indian-English model, in two passes (grammar-constrained, then free), combined by an intent parser: navigate to, search, start/stop navigation, repeat, start/stop awareness, status, help, test voice. Spoken destinations are only proposed and confirmed by the user.
- **Routing:** A* [56] with a haversine heuristic on an OpenStreetMap [50] extract of Vellore (31,439 path nodes, 813 named places). Turn instructions are spoken about 70 m and 25 m ahead; the route is recalculated after two consecutive off-route positions.

- **Optional cloud description:** one photo to Gemini [46] after explicit consent, with a 30 s attempt timeout, a 45 s budget and one retry for a quick "service unavailable"; while waiting, the phone says that local guidance continues.

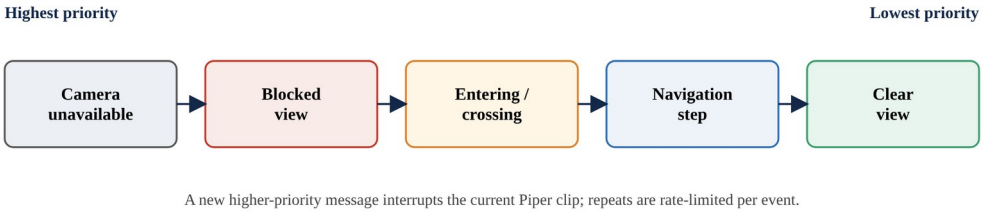


Fig. 14. Priority order of spoken messages.

Description. Messages are ordered from the most urgent (camera unavailable and blocked view) to the least urgent (clear view). A higher-priority message interrupts the clip being played, and repeats of the same event are rate-limited, so the user is not flooded with speech.

Chapter 17: Experimental Setup

Table 15. Test environment

Item	Value
Laptop	HP Omen; Ryzen 7 7840HS (8 cores / 16 threads); 15.3 GB RAM; RTX 4050 Laptop GPU 6 GB; driver 592.82
Software	Windows 11; Python 3.12.10; PyTorch 2.15.0+cu132; torchvision 0.29.1; rfdetr 1.11.2; OpenCV; Piper 1.8.0; Vosk
Environments	`.venv` (CPU only, original); `.venv-gpu` (CUDA)
Detector quality data	365 near-IR validation images, 129 people (WOAD dataset)
Live-pipeline input	640×480 clip compositing a recorded person riding a bicycle over a recorded empty corridor; played as Chrome's fake camera or streamed by a soak-test tool
Browser tests	headless Chrome, iPhone-sized viewport, fake camera and microphone, DNS blocked; server on loopback

No measurement includes Wi-Fi transmission, iPhone JPEG encoding, real street scenes or users.

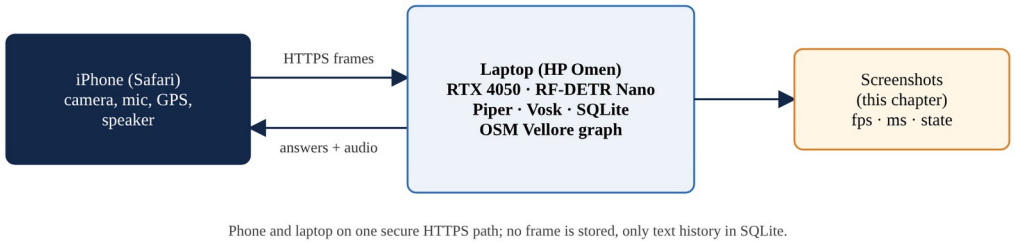


Fig. 15. Test set-up used for the iPhone checks.

Description. The iPhone acts as camera, microphone, GPS and speaker and sends frames over HTTPS to the laptop, which returns the path state, detections and audio. The laptop runs RF-DETR Nano, Piper, Vosk and the offline map. The status line of the phone interface is the source of the readings in Section 18.8.

Chapter 18: Results and Analysis

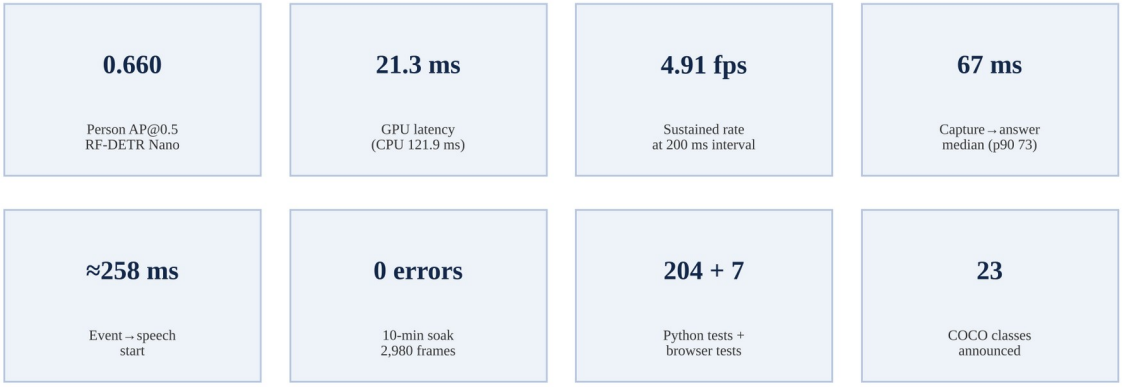


Fig. 16. Headline results of the NeuroSonic prototype.

Description. The tiles summarise the main measured results of this chapter: person AP@0.5 of 0.660 for RF-DETR Nano, 21.3 ms GPU latency, about 4.9 fps sustained, a median 67 ms capture-to-answer time on the laptop, about 258 ms from event to speech, a ten-minute soak without errors, and 204 Python plus 7 browser tests. They are laptop results; iPhone readings follow in Section 18.8.

18.1 Detector comparison

Table 16. *Detector comparison. Latency: mean per 640×480 frame, FP32, batch 1. Quality: person, 365 near-IR images, 129 people*

Detector	GPU ms	CPU ms	AP@0.5	AP@0.5:0.95	Precision @0.40	Recall @0.40	Licence
SSDLite320 (previous)	42.3	43.7	0.542	0.208	0.607	0.574	BSD-3
RF-DETR Nano (selected)	21.3	121.9	**0.660**	**0.304**	0.588	**0.729**	Apache- 2.0
YOLO26n	12.4	27.8	0.619	0.268	0.633	0.628	AGPL-3.0
YOLO26s	11.8	57.5	0.628	0.292	0.600	0.651	AGPL-3.0
WOAD exp72 (reference, NIR + depth)	—	≈126–131*	0.218	0.073	—	—	see repo

* WOAD: CPU forward pass per image in batches of 8, FP32, with its own two-input preprocessing; its AP uses both modalities on the same 365 validation images (see the base-paper re-implementation).

Table 16, Fig. 17 and Fig. 18 summarise the comparison. RF-DETR Nano had the best AP and recall. Recall matters most for a warning system because a missed person is worse than an extra caution. Its 21 ms GPU latency fits a 200 ms frame interval and its licence is permissive. The YOLO26 models are faster but have lower recall and are AGPL-3.0. RF-DETR is too slow on the CPU (122 ms), so SSDLite remains the fall-back. RF-DETR's precision at 0.40 (0.588) is slightly lower than the others', so more false detections are expected; the temporal confirmation is designed to absorb them. On the same images, all four RGB detectors scored far above the WOAD checkpoint, which was trained on different scenes.

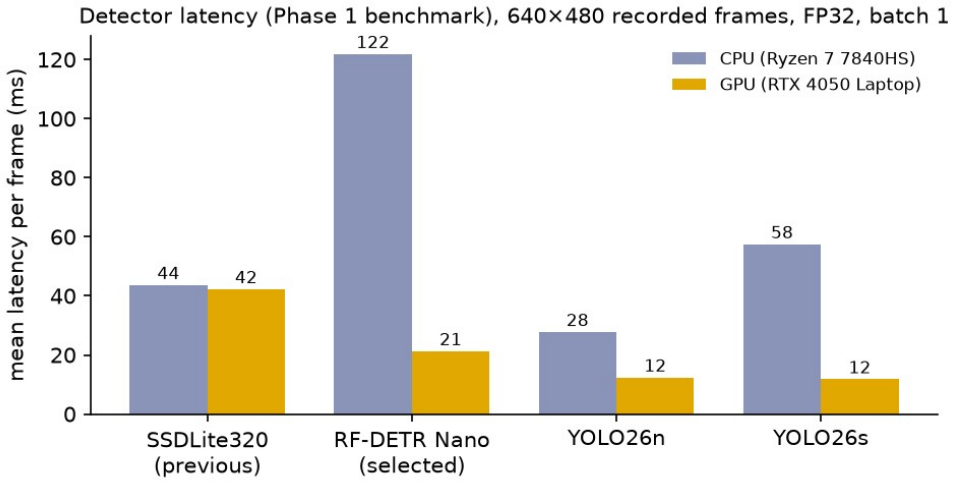


Fig. 17. *Detector latency on GPU and CPU (mean ms per frame).*

Description. The bars compare the mean time per frame of each detector on the GPU and on the CPU. The GPU path is several times faster than the CPU path, which is why RF-DETR Nano is used when a CUDA GPU is present and SSDLite is kept only as a fall-back.

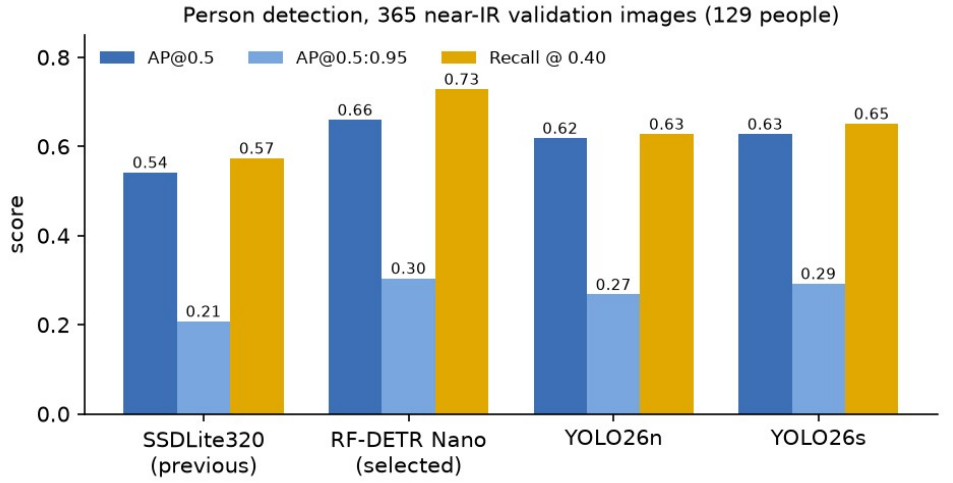


Fig. 18. *Person detection quality on the near-IR validation set.*

Description. The bars compare person detection quality of the candidate detectors on the near-infrared validation set. RF-DETR Nano gives the highest person AP@0.5 and recall, which together with its latency was the basis for choosing it.

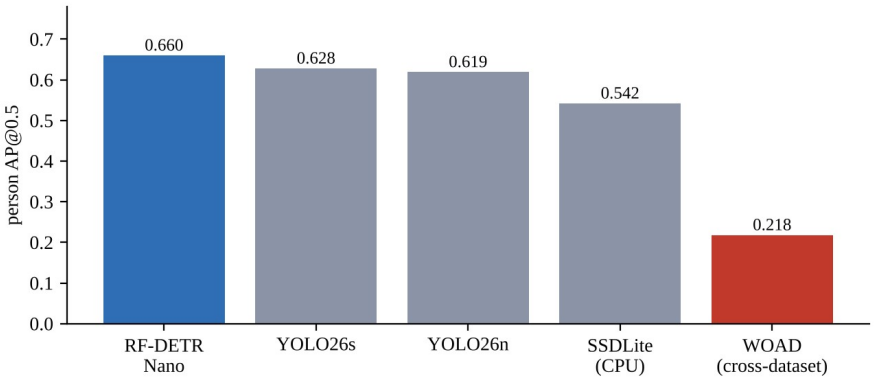


Fig. 19. Person AP@0.5 of the four detectors and of the base-paper model (cross-dataset).

Description. RF-DETR Nano is highest at 0.660, followed by YOLO26s (0.628), YOLO26n (0.619) and SSDLite (0.542), while the WOAD checkpoint re-measured on the published data reaches only 0.218. The chart therefore shows both the reason for choosing RF-DETR Nano and the size of the gap to the base paper.

18.2 FP16 inference

Table 17. *RF-DETR Nano, FP32 vs FP16 (RTX 4050 Laptop GPU, 320×240 frames, 100 timed frames each)*

Precision	AP@0.5	Recall @0.40	Back-to-back med./p90 ms	At 5 fps med./p90 ms
FP32	0.660	0.729	17.3 / 21.7	66.5 / 78.6
FP16 (used)	0.660	0.729	16.9 / 20.4	53.2 / 60.8

Person quality is unchanged in FP16 (AP@0.5:0.95 0.304 vs 0.305; precision at 0.40 0.588 vs 0.595). Back to back both take about 17 ms. At the application's 5 frames/s, however, the GPU drops to its lowest power state between frames (sampled clocks 210–405 MHz), and FP16 lowers the median from 66.5 to 53.2 ms and the p90 from 78.6 to 60.8 ms (Table 17).

18.3 Serving efficiency at a fixed 5 frames/s

CPU threads. With RF-DETR on the GPU, PyTorch's default pool of 8 CPU threads kept the server at 712 % of one CPU core while streaming 5 frames/s, although the GPU did the detection. Two threads cut this to 121 % with no loss of frame rate (Fig. 20); median frame times were 30, 38 and 25 ms for 8, 4 and 2 threads.

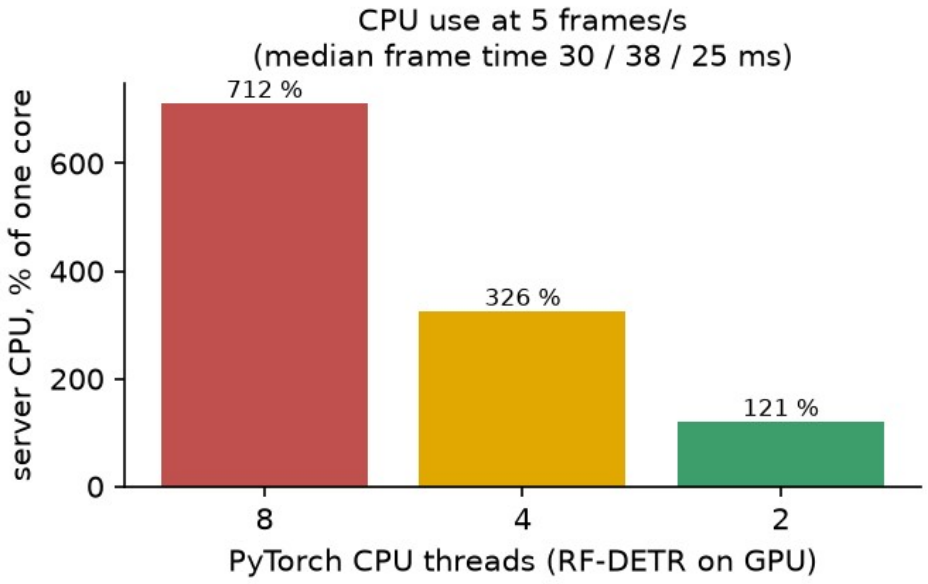


Fig. 20. *Server CPU use with RF-DETR on the GPU, by PyTorch CPU thread count (712, 326, 121 % of one core).*

Description. The chart shows server CPU use at three PyTorch thread settings. Reducing the thread count cut CPU use from 712 % to 121 % of one core with no loss of frame rate, which is why the serving configuration limits the thread count.

GPU power state. At 5 frames/s the GPU is idle about 90 % of the time. In repeated timings it dropped to its lowest performance state (P8, about 390 MHz); the same inference took about 16 ms at P5 (≈600 MHz) and about 74 ms at P8. This explains why live latency is several times the back-to-back benchmark. **Inference thread.** Running GPU inference from a fresh thread per request cost

about 3 ms per call in isolation (19.4 vs 16.2 ms median), so all inference was moved to one long-lived thread.

18.4 Ten-minute soak tests

A soak-test tool streamed the clip to the server at 5 frames/s, one request at a time as the phone does, and every 60 s sent a parallel cloud request to a mock provider that takes 20 s. "Before" ran the earlier code (FP32, inference on request threads); "after" ran the final code.

Table 18. *Ten-minute soak tests, 59 windows of 10 s*

Measure	Before	After
Frames sent / errors	2980 / 0	2980 / 0
Mean frames/s	4.97	4.97
Window medians, range	62–104 ms	70–78 ms
Median of window medians	92.7 ms	74.8 ms
Highest window p90	111.3 ms	92.4 ms
Server RAM, min–max	851–2426 MiB	2255–2431 MiB
Server CPU, median (% of one core)	142	130
Parallel 20 s cloud requests answered	7 / 7	7 / 7
Events spoken	0*	148

* The first run played the clip frame by frame, so the cyclist moved at half speed (0.10 widths/s), below the 0.12 threshold; the event counts are therefore not comparable.

Both runs were stable (Table 18, Fig. 21): no errors, a constant 4.97 frames/s, every slow cloud request answered while frames continued, and no growth in GPU memory. After the changes, latency was steadier (window medians 70–78 ms instead of 62–104 ms) and the worst p90 fell from 111 to 92 ms. Of the 148 events, 100 named the person and 48 the bicycle, which is correct for a person riding a bicycle. Ten minutes on one machine cannot exclude a slow leak over hours.

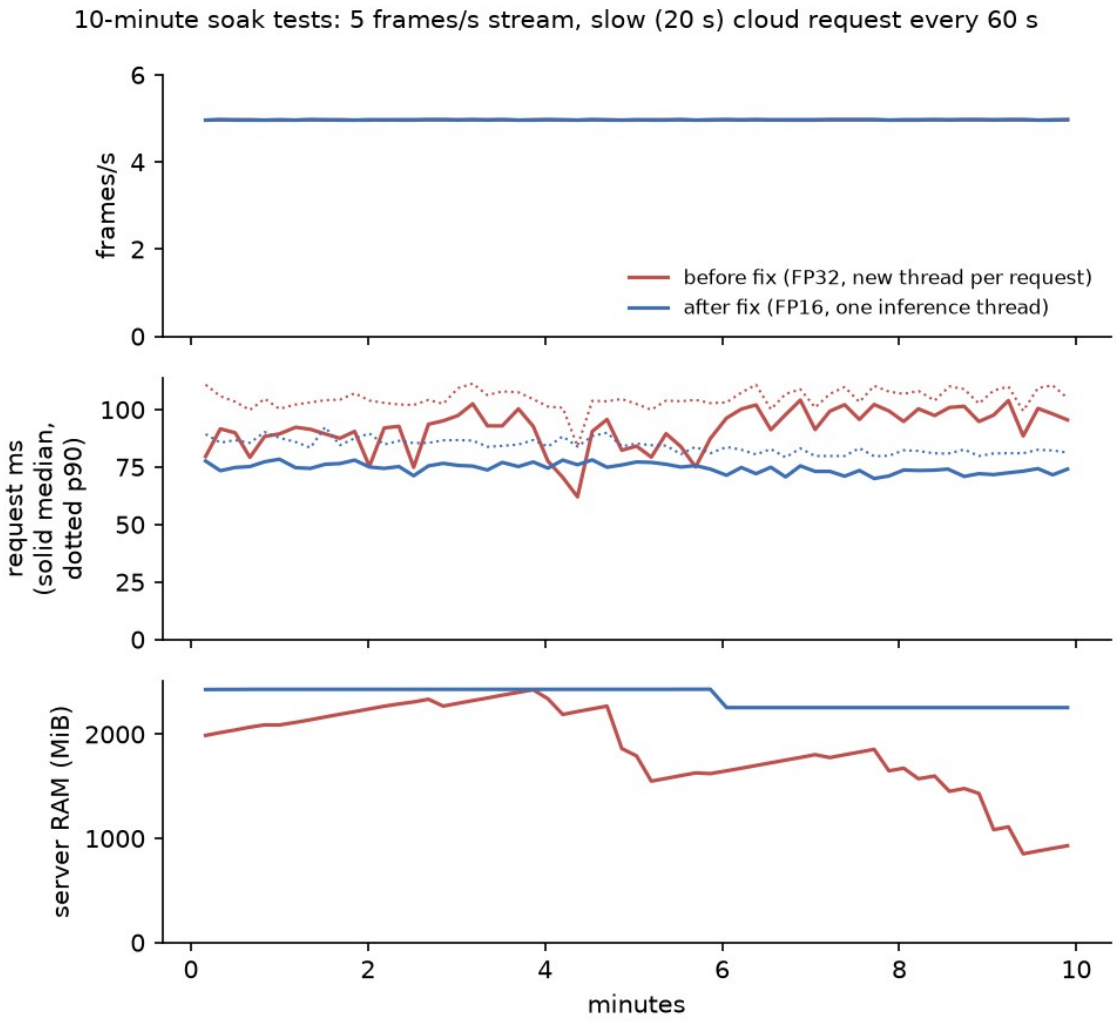


Fig. 21. *Soak tests before and after the serving changes: frames per second, request latency and server memory over ten minutes.*

Description. The plots compare ten-minute soak tests before and after the serving changes, showing frames per second, request latency and server memory. After the changes the rate is steady, the latency is low and flat, and memory does not grow, which indicates a stable server.

18.5 End-to-end behaviour in the browser

Seven headless-Chrome tests drive the real phone interface at iPhone size against the server, with the network blocked. In the event test the page achieved 4.91 frames/s at the recommended 200 ms interval; the capture-to-answer time measured in the page (JPEG encoding, HTTP on loopback, full server pipeline) had a median of 67 ms, p90 73 ms and maximum 81 ms over 44 frames. "Caution. A person is moving into your path from the left." was delivered, and the neural-voice audio started 257 ms later. In the slow-cloud test the phone said "Still waiting for the online description. Local walking guidance continues." at 15.2 s and spoke the description at 20.1 s, while 99 frames were analysed in the meantime. Fig. 22 shows the interface.

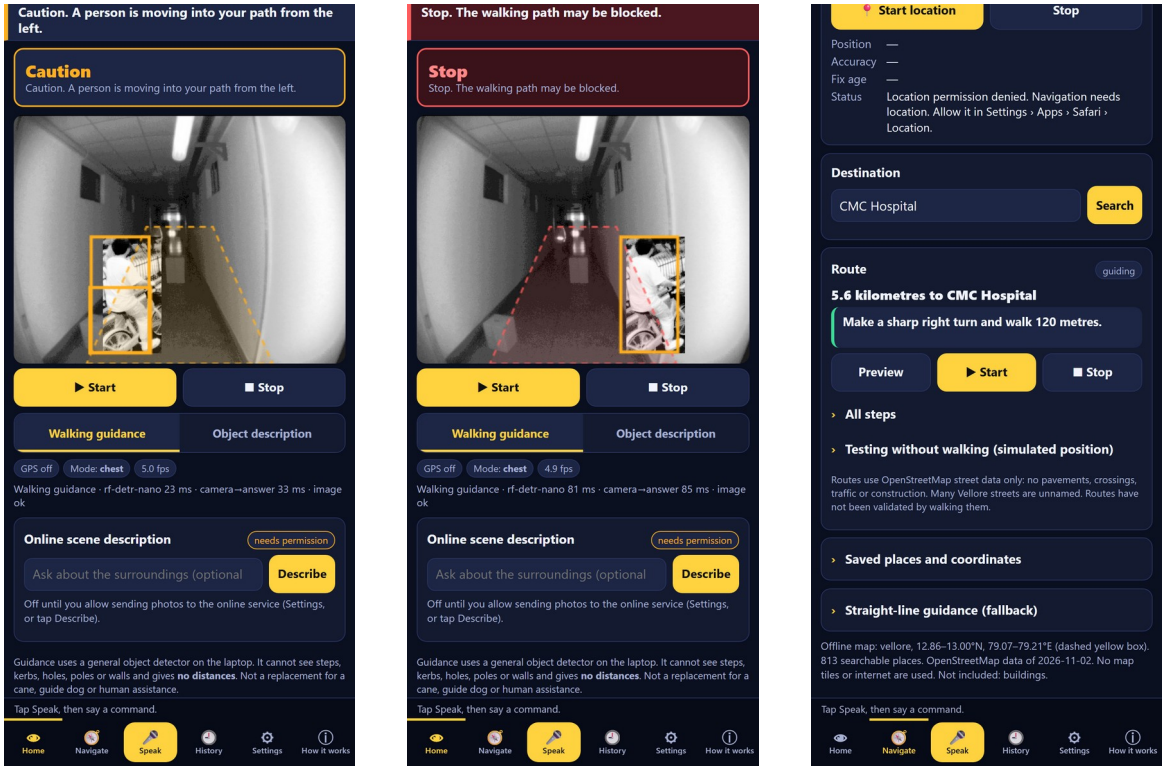


Fig. 22. *Phone interface in Chrome's iPhone-sized emulation with the fake camera. Left: entering event. Centre: blocked path. Right: offline navigation with a simulated position. The camera images are a composite of recorded footage, not field footage.*

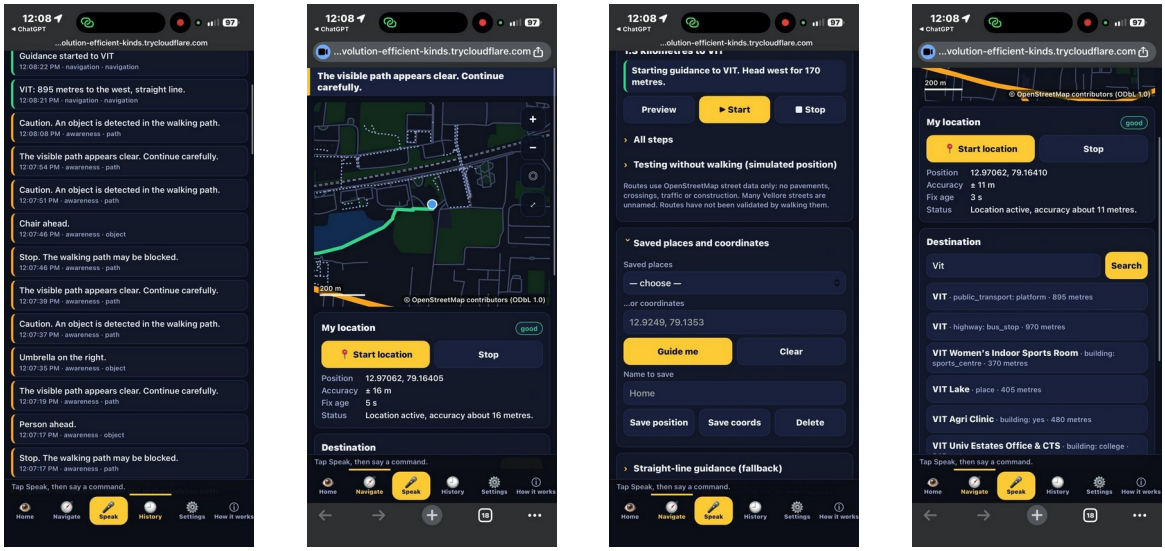
Description. The three panels show the phone interface in Chrome's iPhone-sized emulation with the fake camera. They show the entering event, the blocked-path state and offline navigation with a simulated position. They confirm the layout and the spoken messages before any real iPhone was used; real-device screenshots follow in Section 18.7.

18.6 Automated tests and quality gate

The suite runs 204 test cases (unit, integration and HTTP API tests, plus 32 Node tests of the browser logic): **OK** in the GPU environment (1 skipped: an opt-in live cloud test) and OK in the CPU-only environment (3 skipped: two GPU-only tests and the live test, with automatic fall-back to SSDLite). The 22 perception tests cover detector fall-back, recovery of a known camera shift and zoom, stable track identities while the camera pans, entering and crossing in both directions, the *absence* of events for a static car during a pan, a shaky camera, a blocked view, slow drift and chairs, and the absence of distance and safety words. All seven browser tests pass. A quality-gate launch with the documented command passed in both environments: frames, speech, place search, a 1330 m route with 10 steps, and a voice command. On 3 November 2026 the release launcher started the server as an independent background process, and its local-network HTTPS address (the one the iPhone uses) answered when checked from the laptop; an iPhone was connected later and the live screenshots are reported in Section 18.7; the formal field validation is still pending.

18.7 iPhone screenshots

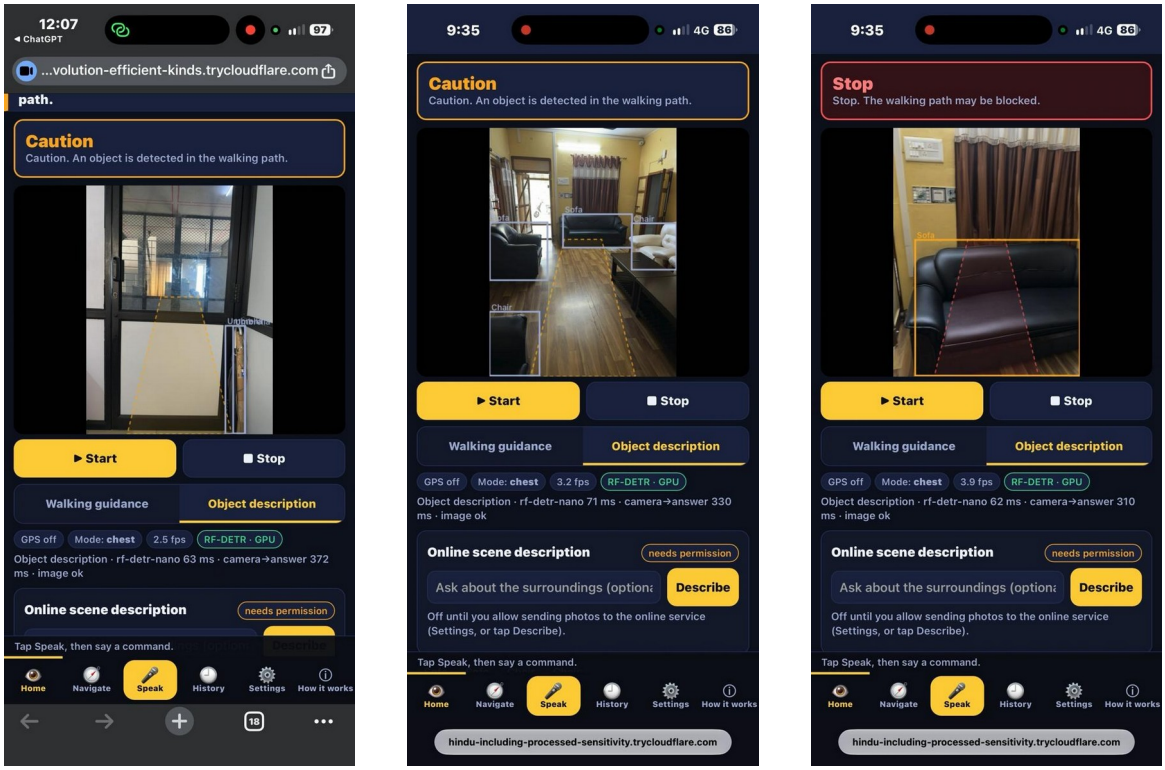
From this point the report uses screenshots taken on a real iPhone running the phone interface in Safari, connected to the laptop server. They are informal checks made while the application was running; they are not the formal 23-scenario × 5-trial validation of Appendix D, which is still pending. Each figure is followed by a description of what every screenshot shows.



(a) History tab (b) Navigate tab with the map (c) Guidance panel (d) Destination search

Fig. 23. Application screens on the iPhone: history, navigation map, route guidance and destination search.

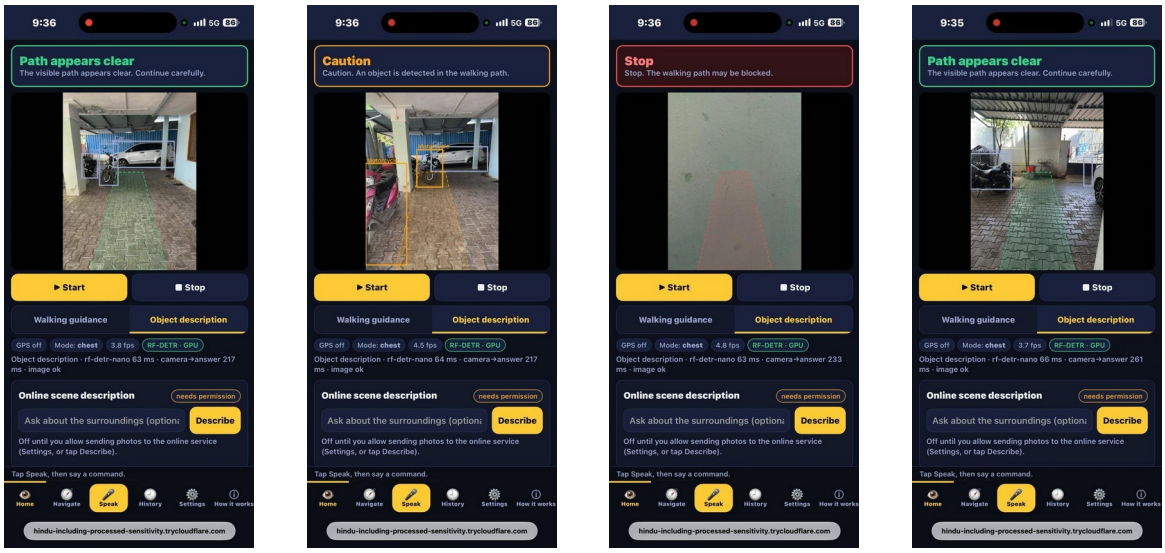
Description. (a) The History tab lists the spoken messages with their time and category: navigation (“Guidance started to VIT”, “VIT: 895 metres to the west, straight line”), path awareness (“Caution. An object is detected in the walking path”, “The visible path appears clear. Continue carefully”, “Stop. The walking path may be blocked”) and object announcements (“Chair ahead”, “Person ahead”, “Umbrella on the right”). Only text is stored. (b) The Navigate tab shows the offline OpenStreetMap map with the route in green, the user's position as a blue dot, and a banner with the current path message; the location panel reads 12.97062, 79.16405 with an accuracy of about 16 m. (c) The guidance panel announces “Starting guidance to VIT. Head west for 170 metres”, with Preview, Start and Stop buttons, the list of all steps, a simulated-position test mode, saved places, and the straight-line fallback; it carries the reminder that routes use OpenStreetMap data only and have not been validated by walking. (d) Searching “Vit” returns places sorted by distance (for example VIT bus platform at 895 m, VIT Lake at 405 m, VIT Agri Clinic at 480 m) and the location panel shows an accuracy of about 11 m.



(a) Caution: doorway (b) Caution: room (c) Stop: sofa and curtain

Fig. 24. Live walking-path guidance on the iPhone in an indoor scene: caution (two views) and stop.

Description. Each screenshot shows the banner at the top, the live camera image with the walking-path trapezoid and detection boxes, and the status line (frame rate, detector and timings). (a) A doorway with a glass partition produces “Caution. An object is detected in the walking path” at 2.5 fps, 63 ms in the detector and 372 ms from camera to answer. (b) A furnished room gives the same caution at 3.2 fps, 71 ms and 330 ms. (c) With a sofa and curtain filling the view, the banner changes to “Stop. The walking path may be blocked” at 3.9 fps, 62 ms and 310 ms. The online scene description is shown as “needs permission”, so no image leaves the local network unless the user allows it.



(a) Path appears clear (b) Caution: parked vehicle (c) Stop: plain wall (d) Path appears clear

Fig. 25. Live walking-path guidance on the iPhone in a courtyard and garage: clear, caution and stop.

Description. (a) A paved courtyard with parked cars at the side is reported as “Path appears clear. Continue carefully” at 3.8 fps, 63 ms and 217 ms. (b) A parked vehicle inside the path trapezoid gives “Caution. An object is detected in the walking path” at 4.5 fps, 64 ms and 217 ms, with the detection boxes drawn on the camera image. (c) Facing a plain, textureless wall and floor, the application shows “Stop. The walking path may be blocked” at 4.8 fps, 63 ms and 233 ms, even though no object class is detected; this is the blocked-view rule at work and is described again in Chapter 20. (d) A garage porch with a motorcycle standing at the side is reported as clear at 3.7 fps, 66 ms and 261 ms.

18.8 Live readings from the iPhone

The status line under each camera image shows the frame rate, the detector time and the camera-to-answer time for the last frame. Table 19 lists these readings for the seven live-camera screenshots and Fig. 26 plots them against the laptop-only results of Section 18.5.

Table 19. *Values read from the status line of the iPhone screenshots (Object description tab, RF-DETR · GPU)*

Screenshot	Scene	Banner shown	fps	Detector (ms)	Camera → answer (ms)
Shot 5	Doorway and glass partition	Caution	2.5	63	372
Shot 6	Furnished room	Caution	3.2	71	330
Shot 7	Sofa and curtain, close	Stop	3.9	62	310
Shot 8	Paved courtyard, parked cars	Path clear	3.8	63	217
Shot 9	Parked vehicle in the path	Caution	4.5	64	217
Shot 10	Plain wall and floor	Stop	4.8	63	233

Screenshot	Scene	Banner shown	fps	Detector (ms)	Camera → answer (ms)
Shot 11	Garage porch, motorcycle at side	Path clear	3.7	66	261
Mean of the seven readings			**3.77**	**64.6**	**277**

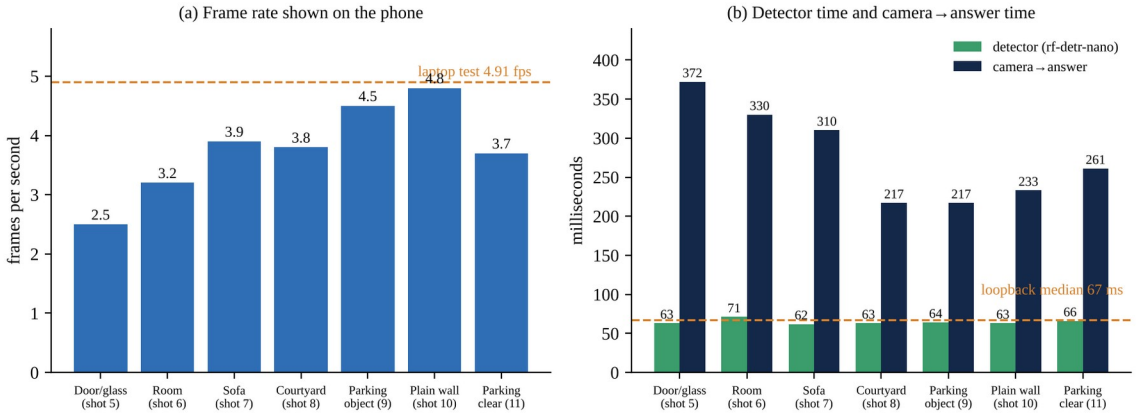


Fig. 26. Frame rate, detector time and camera → answer time read from the seven iPhone screenshots.

Description. Panel (a) shows the frame rate on the phone, between 2.5 and 4.8 fps (mean 3.77 fps), below the 4.91 fps of the laptop test; the rate rises as the scene gets simpler. Panel (b) shows the detector time, which stays at about 62–71 ms (mean 64.6 ms), and the camera → answer time of 217–372 ms (mean 277 ms), which includes the Wi-Fi transfer and the JPEG encoding that the loopback tests did not include.

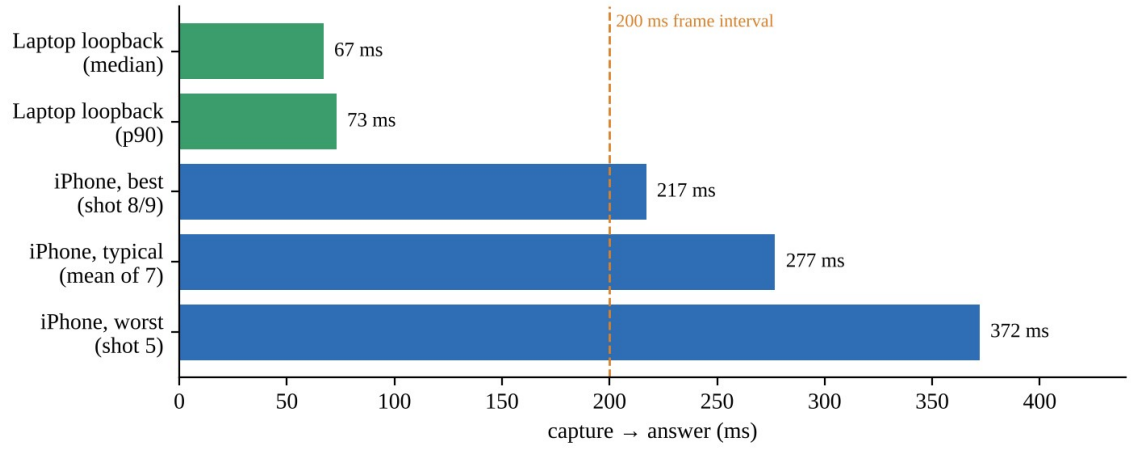


Fig. 27. Capture-to-answer time on the laptop loopback and on the iPhone.

Description. The two green bars are the loopback measurements of Section 18.5 (median 67 ms, p90 73 ms) and the blue bars are the iPhone readings (best 217 ms, mean 277 ms, worst 372 ms). The extra time is the real network and device path. All iPhone values are still above the 200 ms frame interval, so the phone sends fewer frames than the 5 fps target; guidance remains usable, but this is the main item to measure in the formal validation.

These readings come from seven screenshots in informal conditions, so they show the order of magnitude and not a statistically sound latency. The on-screen detector time is the server's per-frame figure as shown by the application and is not directly comparable with the 21.3 ms detector-only benchmark of Table 16.

Chapter 19: Discussion

Table 20. Objectives and their status

Objective	Status	Evidence
O1 real time	met on the laptop	4.91–4.97 frames/s; capture→answer median 67 ms
O2 detector	met within data limits	best AP and recall of four; SSDLite fall-back
O3 motion	partly shown	synthetic-pan tests; composited clip
O4 offline	met in automated tests	network blocked in-process and in Chrome
O5 honesty	met	fixed vocabulary; wording filter; tests
O6 verification	met for the laptop; field pending	204 tests, 7 browser tests, soak tests

Serving details mattered more than the detector's benchmark speed. A detector that takes 21 ms back to back took several times longer at 5 frames/s, because of idle CPU threads, per-request threads and, above all, GPU clock scaling at low load. Benchmarks for assistive systems should therefore be run at the application's frame rate, not only back to back.

Motion without false alarms has a price. In tests with a synthetically panning camera, a static car gave no event while a person moving into the path did. But events need about one second of confirmation and are suppressed while the camera sways strongly, which can include normal walking with a hand-held phone. Missed events are therefore expected; this is acceptable only because the system supplements the cane.

Comparison with the base paper. WOAD reports field results with users and a custom device [3]; NeuroSonic reports laptop and browser results with a phone-based design that any user with a phone and a laptop could run. The two are complementary: WOAD shows what dedicated depth hardware can achieve in the field, while NeuroSonic removes the hardware barrier and adds motion events, navigation and speech input, but has yet to be validated in the field.

Chapter 20: Limitations and Pending Validation

- **Limited iPhone evidence, no formal field validation.** RF-DETR has processed real iPhone frames only in informal screenshots (Section 18.7), and Wi-Fi latency, outdoor light, and use by visually impaired people are untested.
- **Proxy data.** Detector quality was measured on near-infrared images with 129 people; live tests use a fake camera and a composited clip.
- **Scope.** 23 COCO classes only: no steps, kerbs, holes, poles, walls or wet floors; no distance and no "approaching" alerts.
- **Dependence.** The phone needs the laptop and local Wi-Fi; iOS Safari stops the camera when the screen locks; the GPU path needs an NVIDIA GPU.
- **Plain surfaces.** In an informal iPhone check a textureless wall produced “Stop” although no object class was detected (Fig. 25c); the blocked-view rule treats an uninformative view as blocked. Its false-alarm rate on real surfaces is unmeasured.
- **Single machine.** All timings come from one laptop and driver.
- **Cloud description.** The single approved live request timed out; cloud quality and latency are unmeasured.

Table 21. *Physical tests still required (validation protocol: 23 scenarios × 5 trials)*

#	Test	Measure
1	RF-DETR on real iPhone frames	frames processed per client, detector in use
2	Wi-Fi capture→answer latency	median / p90 ms
3	iPhone audio, microphone, voice commands	playback, recognition success
4	Real scenes with cane and sighted companion	events, misses, false alarms
5	GPS route walk in Vellore	turn timing, re-routing
6	Physically disconnected internet	all local functions
7	Live cloud description (with approval)	latency, wording
8	≥ 1 h run over Wi-Fi	errors, memory, temperature

Chapter 21: Conclusion

NeuroSonic shows that a consumer laptop GPU can give a browser-based phone client real-time, offline, spoken awareness of objects and of movement into the walking path, together with offline navigation and voice commands. In our tests it sustained about 5 frames/s and answered within roughly 50–100 ms on the laptop. The general lessons are to measure at the application's frame rate, to constrain CPU threads when the GPU does the work, and to keep GPU work on one thread. The next step is not a new model but evidence: the iPhone and field protocol, followed by monocular depth for "approaching" alerts, ground-hazard data under open licences, and a native application with on-device inference that removes the laptop dependency. NeuroSonic is a research prototype and not a safety device.

Chapter 22: Project Demonstration: Existing Implementation and Proposed Work

The demonstration separates three things: (A) what the base paper provided, (B) what this project implemented and can demonstrate now, and (C) what is proposed but not yet implemented or not yet physically verified.

Table 22. *(A) Base work received (not this project's contribution)*

Item	State when received
Cross-modal detector (NIR + depth), exp72 checkpoint	scripts broken with current libraries
Reported accuracy mAP@0.5 0.669	training plot only; not reproducible with published data
Published dataset	a different dataset (other classes; none of the run's validation frames)
FPGA, RL compression, Android app, SDK	not used (SDK only for the colour-map finding)

Table 23. *(B) Implemented in this project and demonstrable now*

#	Contribution	Verification level
B1	Faithful standalone WOAD inference (bit-identical)	automated tests
B2	Dataset audit and evaluation (person AP@0.5 0.218 / 0.173)	measured
B3	Alert engine and input-quality gate (replay of recorded NIR + depth)	automated tests
B4	NeuroSonic server, API (28 endpoints), SQLite text history, accessible phone UI	automated + browser tests
B5	Detector benchmark; RF-DETR Nano FP16 on GPU, SSDLite CPU fall-back	measured; tests
B6	Camera-motion-compensated tracking; entering / crossing events	tests; composited clip
B7	Piper neural voice; priority speech queue	browser tests (laptop)
B8	Vosk offline voice commands	synthetic speech; browser test with fake mic
B9	Offline Vellore map, search, A* routing, spoken turns, re-routing	tests; simulated walk
B10	HTTPS with local certificate authority; phone setup page; release start/stop scripts	tests; launch verified
B11	Optional opt-in cloud description with wording filter	mock-provider tests
B12	Real iPhone Safari client with live camera, path banners and status line (informal screenshots)	screenshots, Section 18.7

Table 24. *(C) Proposed work, not yet implemented or not yet physically verified*

#	Proposal	Reason
C1	iPhone and field validation (23 scenarios × 5 trials)	only informal iPhone screenshots so far; the formal protocol has not been run
C2	Monocular depth for "approaching" alerts	no distance from an RGB camera today
C3	Ground-hazard data: steps, kerbs, potholes, poles	outside the 23 COCO classes
C4	Native app with on-device inference	removes the laptop dependency
C5	Better recognition of spoken place names	10 of 24 synthetic utterances correct first

#	Proposal	Reason
C6	User study with visually impaired participants (with ethics approval)	thresholds and wording are engineering defaults

22.1 Demonstration script (about 10 minutes)

1. **Base paper (2 min).** Run the re-implemented WOAD detector on a recorded NIR + depth sequence (Settings → Research tools → WOAD replay); show the equivalence test result and the evaluation table.
2. **Launch (1 min).** `START_NEUROSONIC.ps1`; show the printed detector (RF-DETR, cuda, fp16) and the iPhone address.
3. **Walking-path awareness (3 min).** On the phone (or the laptop browser with the demo clip), press Start; show "appears clear", "Caution", "Stop", and the event "A person is moving into your path from the left"; pan the camera over a static scene to show that no event is raised.
4. **Voice and navigation (2 min).** Say "navigate to CMC Hospital"; confirm; show the offline route and spoken turn instruction; show that it works with the internet disconnected.
5. **Honesty and limits (2 min).** Show the "How it works" screen; state what is not yet verified.

Appendix A: API Endpoints

Table 25. HTTP API of the laptop server (16 GET, 12 POST)

Method	Path	Purpose
GET	`/api/status`	server, model and feature status
GET	`/api/events`	recent alert events
GET	`/api/replay/state`, `/api/replay/frame.jpg`	WOAD replay state and current frame
GET	`/api/history`, `/api/stats`	text history and statistics
GET	`/api/places`	saved places
GET	`/api/tts/status`, `/api/tts`	neural voice status; synthesised WAV (range requests)
GET	`/api/voice/reports`	playback reports from phones
GET	, , `/api/map/info` `/api/map/data` `/api/map/search`	offline map metadata, drawing data, place search
GET	`/api/voicecmd/status`	offline speech-recognition status
GET	`/api/awareness/status`	detector, precision, weights, per-client statistics
GET	`/api/vision/status`	optional cloud-description status
POST	`/api/replay/start`, `/api/replay/stop`	control WOAD replay
POST	`/api/detect`	WOAD detection on an uploaded NIR + depth pair
POST	`/api/alerts`	record client-side alerts in the history
POST	`/api/history/clear`	clear history
POST	, `/api/places` `/api/places/delete`	save / delete a place
POST	`/api/voice/report`	phone reports how a clip was played
POST	`/api/route`	offline A* walking route
POST	`/api/voicecmd`	recognise a spoken command (WAV)
POST	`/api/awareness/frame`	analyse one camera frame (JPEG)
POST	`/api/vision/describe`	optional cloud scene description (opt-in)

Appendix B: Software, Models and Licences

Table 26. Main third-party components

Component	Use	Licence
PyTorch, torchvision	inference	BSD-style
RF-DETR (rfdetr 1.11.2), Nano weights	primary detector	Apache-2.0
SSDLite320-MobileNetV3 (torchvision)	CPU fall-back	BSD-3
YOLO26 (ultralytics)	benchmark only	AGPL-3.0
OpenCV	optical flow, RANSAC	Apache-2.0
Piper 1.8.0; voice en_IN-spicor	speech synthesis	GPL-3.0+; voice AGPL-3.0
Vosk, small Indian-English model	speech recognition	Apache-2.0
OpenStreetMap data (Vellore extract)	map and routing	ODbL 1.0
COCO annotations	detector training (third party)	CC BY 4.0
WOAD code, checkpoint, dataset	base paper, research path	see original repository

The GPL/AGPL voice components constrain redistribution and must be reviewed before any commercial release.

Appendix C: Installation, Launch and Reproduction

```
cd F:\Projects\WOAD-Research\implementation
powershell -ExecutionPolicy Bypass -File .\START_NEUROSONIC.ps1 # GPU (RF-DETR); add -Cpu
for SSDLite
powershell -ExecutionPolicy Bypass -File .\STOP_NEUROSONIC.ps1
# manual (foreground)
..\.venv-gpu\Scripts\python neurosonic_app.py --host 0.0.0.0 --https
# tests
..\.venv-gpu\Scripts\python -m unittest discover -s tests -t .
# results used in this report
outputs\benchmarks\phase1\results.json detector comparison
outputs\benchmarks\precision\results.json FP32 vs FP16
outputs\benchmarks\soak\*.json 10-minute soak tests
outputs\eval\nc10_*_person\metrics.json WOAD evaluation
tools\make_publication_figures.py figures
```

On the iPhone, the start script prints the address to open in Safari (on 3 November 2026: ,`https://192.168.1.5:8765` with the one-time certificate setup page at). The address depends on the Wi-Fi network.`http://192.168.1.5:8766`

Appendix D: Physical Validation Protocol (Summary)

None of these tests has been performed yet. Each scenario is repeated five times; every trial is recorded, including failures and missed alerts; latency is measured, not estimated. Safety rules: the tester walks with a sighted companion or their usual mobility aid; vehicles are observed only from a safe place; NeuroSonic is never relied on.

Table 27. *Validation scenarios*

Code	Scenario	Pass criterion (summary)
P1	Person walks into the path from the side	entering or caution message
P2	Person approaching head-on	"Caution", then "Stop" before arm's reach
P3	Person crossing 6–10 m ahead	crossing message with correct direction, or none
V1	Car passing (observed safely)	no false "Stop" for cars outside the path
V2	Motorcycle / bicycle	correct class
S1	Static obstacle in the path	caution/stop; no "moving into your path"
S2	Obstacle at the side	no "Stop"
W1	Walking, chest-mounted, corridor with people	few false events per minute
W2	Hand-held, phone swinging	events suppressed
L1–L3	Daylight; dim light; lens covered	as P1; "cannot clearly assess" when dark; recovery
B1	Blur	uncertain rather than "appears clear"
R1, R2	Alert latency; interruption of navigation speech	≤ 2 s target; interruption and replay
G1–G3	GPS fix; short route walk; off-route	turns at about 70 m and 25 m; re-route
N1, N2	Wi-Fi drop; internet off, Wi-Fi on	clear messages; local functions keep working
C1, C2	Cloud description; slow cloud	result or clear failure; guidance continues
T1	15-minute session	no crash; about 5 frames/s; battery and temperature noted

References

[1] S. Real and A. Araujo, "Navigation systems for the blind and visually impaired: past work, challenges, and open problems," *Sensors*, vol. 19, no. 15, Art. no. 3404, 2019, doi: 10.3390/s19153404.

[2] B. Kuriakose, R. Shrestha, and F. E. Sandnes, "Tools and technologies for blind and visually impaired navigation support: a review," *IETE Technical Review*, 2020, doi: 10.1080/02564602.2020.1819893.

[3] Y. Gao, D. Wu, J. Song, X. Zhang, B. Hou, H. Liu, J. Liao, and L. Zhou, "A wearable obstacle avoidance device for visually impaired individuals with cross-modal learning," *Nature Communications*, vol. 16, Art. no. 2857, Mar. 2025, doi: 10.1038/s41467-025-58085-x.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: unified, real-time object detection," in *Proc. IEEE CVPR*, 2016.

[5] N. Carion *et al.*, "End-to-end object detection with transformers," in *Proc. ECCV*, 2020.

[6] T. Ghahremanians and H. Mahvash Mohammadi, "Independent pathfinding with collision avoidance for visually impaired individuals," *Scientific Reports*, vol. 15, Art. no. 35441, Oct. 2025, doi: 10.1038/s41598-025-19387-8.

[7] Tang *et al.*, "Human-centred design and fabrication of a wearable multimodal visual assistance system," *Nature Machine Intelligence*, vol. 7, pp. 627–638, Apr. 2025, doi: 10.1038/s42256-025-01018-6.

[8] D. K. Creasy, J. Karr, M. Klarer, R. Meredith, L. Skutchan, and J. Wegner (American Printing House for the Blind), "Navigation assistance for the visually impaired," U.S. Patent 11 318 050 B2, May 3, 2022.

[9] B. Kimia (Brown University), "Navigation methods and apparatus for the visually impaired," U.S. Patent 11 112 261 B2, Sep. 7, 2021.

[10] W. Liu *et al.*, "SSD: single shot multibox detector," in *Proc. ECCV*, 2016.

[11] A. Howard *et al.*, "Searching for MobileNetV3," in *Proc. IEEE ICCV*, 2019.

[12] Perkins School for the Blind, "Visual assistance apps post roundup." [Online]. Available: https://www.perkins.org/resource/visual-assistance-apps-post-roundup/

[13] World Health Organization, "Blindness and vision impairment," Fact sheet, 10 Feb. 2026. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment

[14] World Health Organization, *World Report on Vision*. Geneva, Switzerland: WHO, Oct. 2019. [Online]. Available: https://www.who.int/news/item/08-10-2019-who-launches-first-world-report-on-vision

[15] GBD 2019 Blindness and Vision Impairment Collaborators and Vision Loss Expert Group of the Global Burden of Disease Study, "Trends in prevalence of blindness and distance and near vision impairment over 30 years: an analysis for the Global Burden of Disease Study," *Lancet Global Health*, vol. 9, no. 2, pp. e130–e143, 2021, doi: 10.1016/S2214-109X(20)30425-3.

[16] National Programme for Control of Blindness and Visual Impairment, Ministry of Health and Family Welfare, Govt. of India, and Dr. R. P. Centre for Ophthalmic Sciences, AIIMS, "National Blindness and Visual Impairment Survey 2015–2019," India Vision Atlas. [Online]. Available: https://indiavisionatlasnpcb.aiims.edu/

[17] Vikaspedia (Govt. of India), "National Blindness and Visual Impairment Survey," summary of the 2015–2019 survey. [Online]. Available: https://vikaspedia.in/health/health-directory/national-blindness-and-visual-impairment-survey

[18] P. Christodoulou, "Soft actor-critic for discrete action settings," arXiv:1910.07207, 2019.

[19] G. Jocher *et al.*, "Ultralytics YOLOv5," software, 2020. [Online]. Available: https://github.com/ultralytics/yolov5

[20] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017.

[21] J. Bai, Z. Liu, Y. Lin, Y. Li, S. Lian, and D. Liu, "Wearable travel aid for environment perception and navigation of visually impaired people," *Electronics*, 2019 (arXiv:1904.13037).

[22] Y. Lin, K. Wang, W. Yi, and S. Lian, "Deep learning based wearable assistive system for visually impaired people," in *Proc. ICCV Workshop on Assistive Computer Vision and Robotics (ACVR)*, 2019 (arXiv:1908.03364).

[23] P. Xu, A. Song, and K. Wang, "Intelligent head-mounted obstacle avoidance wearable for the blind and visually impaired," *Sensors*, vol. 23, no. 23, Art. no. 9598, Dec. 2023, doi: 10.3390/s23239598.

[24] F. Yan, T. Zeng, and M. He, "Implementing blind navigation through multi-modal sensing and gait guidance," arXiv:2506.19593, Jun. 2025.

[25] X. Li *et al.*, "AviPer: assisting visually impaired people to perceive the world with visual-tactile multimodal attention network," *CCF Trans. Pervasive Computing and Interaction*, 2022.

[26] J. Guerreiro, D. Sato, S. Asakawa, H. Dong, K. M. Kitani, and C. Asakawa, "CaBot: designing and evaluating an autonomous navigation robot for blind people," in *Proc. 21st Int. ACM SIGACCESS Conf. Computers and Accessibility (ASSETS)*, 2019, pp. 68–82, doi: 10.1145/3308561.3353771.

[27] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE ICIP*, 2016.

[28] Y. Zhang *et al.*, "ByteTrack: multi-object tracking by associating every detection box," in *Proc. ECCV*, 2022.

[29] G. M. Elchinger (Polaroid Corp.), "Mobility cane for the blind incorporating ultrasonic obstacle sensing apparatus," U.S. Patent 4 280 204, Jul. 21, 1981.

[30] W. Fink and M. Humayun, "Digital object recognition audio-assistant for the visually impaired," U.S. Patent Appl. Publ. 2005/0208457 A1, Sep. 22, 2005 (abandoned).

[31] C. Searle, S. Searle, and J. Holbrook, "Navigation device for the visually-impaired," U.S. Patent 9 429 446 B1, Aug. 30, 2016.

[32] F. D'Angelo and S. Corona (STMicroelectronics S.r.l.), "Wearable obstacle-detection device, and corresponding method and computer program product," U.S. Patent 9 846 231 B2, Dec. 19, 2017.

[33] T. Chen and R. F. Tow (Toyota Motor Engineering and Manufacturing North America), "Vision-assist systems and methods for assisting visually impaired users with navigating an environment using simultaneous audio outputs," U.S. Patent 9 993 384 B1, Jun. 12, 2018.

[34] D. M. Frankel, "Systems and methods for blind and visually impaired person environment navigation assistance," U.S. Patent 10 299 982 B2, May 28, 2019.

[35] D. Bell, S. Houts, L. V. Keiser, J. Oh, and G. Pandey (Ford Global Technologies), "Navigational aid for the visually impaired," U.S. Patent 10 809 079 B2, Oct. 20, 2020.

[36] AARP, "Low vision technology" (product overview incl. OrCam MyEye Pro and WeWALK prices). [Online]. Available: https://www.aarp.org/health/conditions-treatments/low-vision-technology/

[37] IAPB Valued Supplier directory, "WeWALK." [Online]. Available: https://valuedsupplier.iapb.org/suppliers/wewalk

[38] Wellcome Trust grant 092882/A/10/Z (SmartCane, IIT Delhi Assistech with Saksham Trust); device description in ThisAbility, "SmartCane." [Online]. Available: https://grantnav.threesixtygiving.org/grant/360G-Wellcome-092882_A_10_Z

[39] RNIB Shop, "OrCam MyEye Pro" and "OrCam MyEye Smart" product pages. [Online]. Available: https://shop.rnib.org.uk/products/orcam-myeye-pro

[40] Thomas Pocklington Trust, "Google Lookout review." [Online]. Available: https://www.pocklington-trust.org.uk/technology/tech-news-and-views/google-lookout-review/

[41] Ophthalmology Times, "Smart harness alerts visually impaired people to obstacles in their paths"; and RealSense, "Biped.ai: real-time guidance for the visually impaired" (case study). [Online]. Available: https://www.ophthalmologytimes.com/view/smart-harness-alerts-visually-impaired-people-to-obstacles-in-their-paths

[42] Popular Science, "Can this robot help solve a guide dog shortage?"; ITU AI for Good, "Meet Glidance." [Online]. Available: https://popsci.com/technology/can-this-robot-help-solve-a-guide-dog-shortage

[43] T.-Y. Lin *et al.*, "Microsoft COCO: common objects in context," in *Proc. ECCV*, 2014.

[44] Roboflow, "RF-DETR: real-time detection transformer," software, Apache-2.0. [Online]. Available: https://github.com/roboflow/rf-detr

[45] M. Oquab *et al.*, "DINOv2: learning robust visual features without supervision," *Trans. Machine Learning Research*, 2024 (arXiv:2304.07193).

[46] Google, "Gemini API documentation." [Online]. Available: https://ai.google.dev/

[47] D. Povey *et al.*, "The Kaldi speech recognition toolkit," in *Proc. IEEE ASRU Workshop*, 2011; Vosk offline speech recognition. [Online]. Available: https://alphacephei.com/vosk/

[48] Piper text-to-speech, software. [Online]. Available: https://github.com/rhasspy/piper

[49] J. Kim, J. Kong, and J. Son, "Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech," in *Proc. ICML*, 2021.

[50] OpenStreetMap contributors, OpenStreetMap (ODbL 1.0). [Online]. Available: https://www.openstreetmap.org

[51] Ultralytics, "YOLO26 documentation," software, AGPL-3.0. [Online]. Available: https://docs.ultralytics.com/

[52] J. Shi and C. Tomasi, "Good features to track," in *Proc. IEEE CVPR*, 1994.

[53] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proc. IJCAI*, 1981.

[54] J.-Y. Bouguet, "Pyramidal implementation of the Lucas Kanade feature tracker," Intel Corp., Microprocessor Research Labs, Tech. Rep., 2000.

[55] M. A. Fischler and R. C. Bolles, "Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography," *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[56] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[57] OpenCV library. [Online]. Available: https://opencv.org/